

Claude Multi-Account Fine Tuning

Running multiple Claude Code accounts on one host with shared history, memory, and settings

milos85vasic

2026-06-28

Claude Multi-Account Fine Tuning

1. Motivation
2. How Claude Code stores state
3. Architecture after fine-tuning
4. Reproducing this on any host
 - 4.0 Clean-slate host: `claude-bootstrap`
 - 4.1 Prerequisites
 - 4.2 Lay down the scripts directory
 - 4.3 Run the installer
 - 4.4 Add a third (or fourth, ...) account
 - 4.5 Verify the result
 - 4.6 Rollback if needed
5. Operational notes
 - 5.1 Concurrent use of two accounts
 - 5.2 Plugin path rewriting
 - 5.3 Adding a new account from scratch (no existing dir)
 - 5.4 Removing an account
 - 5.5 Backups created during install
6. Cross-platform reproduction
 - Linux
 - macOS
 - WSL / Windows Subsystem for Linux
7. Tests
 - 7.1 Running the suite
 - 7.2 What each test covers
 - 7.3 How the test harness works
 - 7.4 Adding a new test
8. The complete script source
 - 8.1 `lib.sh`
 - 8.2 `claude-unify.sh`
 - 8.3 `claude-add-account.sh`
 - 8.4 `claude-remove-account.sh`

- 8.5 `claude-list-accounts.sh`
- 8.6 `claude-rollback.sh`
- 8.7 `claude-export-docs.sh`
- 8.8 `install.sh`
- 8.9 `tests/lib/assert.sh`
- 8.10 `tests/lib/sandbox.sh`
- 8.11 `tests/run-all.sh`
- 8.12 `tests/test_lib.sh`
- 8.13 `tests/test_unify.sh`
- 8.14 `tests/test_add_remove.sh`
- 8.15 `tests/test_list.sh`
- 8.16 `tests/test_export.sh`
- 11. Provider aliases — running other LLMs through Claude Code
 - Data flow (nothing hardcoded)
 - Transports
 - Safety
 - Commands
 - Session color — auto-applied (v1.10.0)
- 12. Per-project auto-session & per-alias color
- 13. TOON utility

Claude Multi-Account Fine Tuning

This document captures, in reproducible detail, how a single host can run multiple Claude Code accounts (e.g. `claude1`, `claude2`, ..., `claudeN`) while sharing conversation history, project memory, todos, plans, plugins, and settings between them. Only account identity (`.credentials.json`, `.claude.json`, MCP auth cache) is kept private per account.

Companion artifacts:

- `~/Documents/scripts/` — automation scripts (the source of truth for the procedure described here).
- `Claude_Multi_Account_Fine_Tuning.html` — same content as this file, rendered as a standalone HTML page.
- `Claude_Multi_Account_Fine_Tuning.pdf` — same content, paginated PDF.

The HTML and PDF are regenerated by `claude-export-docs.sh`, so editing the markdown is the only place changes need to be made.

1. Motivation

The Claude Code CLI is invoked as `claude`. By default it writes all of its state — credentials, conversation transcripts, project memory, todos, plans, plugin installs, settings — under a single config directory (`~/.claude` on Linux/macOS).

When you want to use **multiple Anthropic accounts** (for example, one personal Max subscription and one team/work account), you typically run two separate config dirs and switch between them with an environment variable:

```
# ~/.bashrc (before this setup)
export CLAUDE_BIN="$HOME/.local/bin/claude"
alias claude1="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic
               $CLAUDE_BIN"
alias claude2="CLAUDE_CONFIG_DIR=$HOME/.claude-milos85vasic2nd
               $CLAUDE_BIN"
```

That works for isolation, but the two accounts now have **completely separate** project memory, history, todos, and plugin sets. Switching accounts means losing the working context you built up under the other.

Goal: keep the two-alias UX (cheap account switch) while ensuring both accounts read and write the same project memory, conversation transcripts, todos, plans, plugins, and settings.

2. How Claude Code stores state

Every key piece of state lives under the directory pointed at by `CLAUDE_CONFIG_DIR` (with `~/.claude` as the default). Items fall into three categories:

Category	Path under CLAUDE_CONFIG_DIR	What it is
Identity (private)	<code>.credentials.json</code>	OAuth tokens for the Anthropic account.
	<code>.claude.json</code>	Account onboarding/migration markers, <code>numStartups</code> , etc.
	<code>mcp-needs-auth-cache.json</code>	Per-account MCP auth state.
Work product (share)	<code>projects/<slug>/<uuid>.jsonl</code>	Per-session conversation transcript.
	<code>projects/<slug>/memory/*.md</code>	

Category	Path under CLAUDE_CONFIG_DIR	What it is
Runtime (share-able with care)		Project memory entries (what Skill auto-memory writes).
	history.jsonl	Prompt-line history (Up/ Down recall).
	todos/, tasks/, plans/	UUID-named files for TaskCreate/Plan tooling.
	file-history/	Snapshots of files edited via the Edit tool.
	paste-cache/	Long-paste buffer storage.
	plugins/	Installed plugins, marketplaces, manifest JSON.
	settings.json	Top-level config: enabledPlugins, effortLevel, etc.
	CLAUDE.md	User-scope memory / instructions.
	sessions/	Active session lockfiles. Not safe to share between concurrent processes.
	telemetry/	Per-account telemetry batches.
	cache/, backups/, session-env/, shell- snapshots/	Ephemeral runtime caches.
	stats-cache.json	Per-account usage / quota stats.

The key observation: every **work-product** item is keyed either by UUID (session, todo, plan, file edit) or by project slug. That means combining two accounts' work-product directories is a **set-union operation**: copying both into the same destination never overwrites either's data, because each file's path is already unique.

The handful of files that genuinely need merging are:

- settings.json — both files have a top-level enabledPlugins map. We union it, then keep the most permissive top-level keys (effortLevel: "high", skipDangerousModePermissionPrompt: true).
- history.jsonl — append-only prompt log. We concatenate both files and de-duplicate by line, preserving order of first appearance.

- `plugins/installed_plugins.json` and `plugins/known_marketplaces.json` — these embed **absolute paths** to plugin code. We rewrite those paths to live under the shared store regardless of which account they came from.
-

3. Architecture after fine-tuning

```
~/.claude-shared/          <-- single source of truth
  projects/                (transcripts + memory; per-slug)
  todos/  tasks/  plans/
  file-history/  paste-cache/
  plugins/
    installed_plugins.json  (paths rewritten -> ~/.claude-
shared/...)
    known_marketplaces.json (paths rewritten -> ~/.claude-
shared/...)
    cache/<marketplace>/<plugin>/<version>/
    marketplaces/<marketplace>/
  shell-snapshots/  session-env/
  telemetry/  sessions/  backups/  cache/
  history.jsonl  settings.json  stats-cache.json
  CLAUDE.md
```

```
~/.claude-milos85vasic/    <-- account 1 (claude1 alias)
  .credentials.json        (PRIVATE – claude1's OAuth tokens)
  .claude.json              (PRIVATE – claude1's onboarding state)
  mcp-needs-auth-cache.json (PRIVATE)
  projects      -> ~/.claude-shared/projects
  todos         -> ~/.claude-shared/todos
  tasks        -> ~/.claude-shared/tasks
  plans         -> ~/.claude-shared/plans
  plugins       -> ~/.claude-shared/plugins
  settings.json -> ~/.claude-shared/settings.json
  history.jsonl -> ~/.claude-shared/history.jsonl
  CLAUDE.md     -> ~/.claude-shared/CLAUDE.md
  ... (every shared item is a symlink)
```

```
~/.claude-milos85vasic2nd/ <-- account 2 (claude2 alias)
  .credentials.json        (PRIVATE – claude2's OAuth tokens)
  .claude.json              (PRIVATE – claude2's onboarding state)
  mcp-needs-auth-cache.json (PRIVATE)
  ... (same symlink set as above)
```

```
~/.claude/                  <-- user-scope default root
  CLAUDE.md      -> ~/.claude-shared/CLAUDE.md
  plugins/
    cache        -> ~/.claude-shared/plugins/cache
    marketplaces -> ~/.claude-shared/plugins/marketplaces
```

Properties of this layout:

1. **Account switching is purely a credentials swap.** Running `claude1` vs `claude2` changes `CLAUDE_CONFIG_DIR`, which changes which `.credentials.json` is loaded — everything else (history, memory, plugins) resolves through symlinks to the same shared store.
 2. **Adding a new account is the same operation, parameterised.** A new `~/.claude-<name>` dir with the same symlink set + its own `.credentials.json` slots in seamlessly. See `claude-add-account.sh`.
 3. **Account identity stays compartmentalised.** OAuth tokens are not in the shared store, so leaking the shared store does not leak account credentials.
 4. **Rollback is mechanical.** Every original directory was moved to `<path>.preunify.<timestamp>` before the symlink replaced it. A single script (`claude-rollback.sh`) reverses the migration.
-

4. Reproducing this on any host

This section is the actual step-by-step procedure. It assumes you have two (or more) Claude Code accounts already set up the conventional way — one config directory per account, each with its own `.credentials.json`. If you don't have a second account yet, run the first three steps and then use `claude-add-account.sh` to create one.

4.0 Clean-slate host: `claude-bootstrap`

If you are setting up a **brand-new machine** where Claude Code is installed but no accounts have been created or logged in yet, use `claude-bootstrap` instead of the §4.2–§4.3 flow. It provisions N fresh accounts in one shot:

```
git clone <this repo> claude-toolkit
cd claude-toolkit
bash scripts/claude-bootstrap.sh --count 2 --yes
exec $SHELL -l
claude1 /login
claude2 /login
```

Key flags:

Flag	Effect
<code>--count N</code>	Create N accounts named <code>claude1...</code> <code>claudeN</code> .
<code>--yes</code>	Non-interactive; skip confirmation prompts.

Flag	Effect
--aliases personal,work	Use custom alias names instead of --count.
--dir-of NAME=PATH	Override the config-dir path for a specific alias.

claude-bootstrap is **idempotent** — safe to re-run after pulling updates. It calls `install.sh` internally, so you do not need to run both.

4.1 Prerequisites

```
# Linux (Debian/Ubuntu/Fedora/Arch)
sudo apt-get install -y rsync jq gawk pandoc weasyprint      #
                        Debian/Ubuntu
# or
sudo dnf install -y rsync jq gawk pandoc weasyprint          #
                        Fedora
# or
sudo pacman -S rsync jq gawk pandoc weasyprint              # Arch
# macOS (Homebrew)
brew install rsync jq gawk pandoc weasyprint
```

weasyprint is optional but produces the best-looking PDF; the export script will fall back to wkhtmltopdf or chromium --headless if weasyprint isn't available.

4.2 Lay down the scripts directory

Clone or copy the entire `scripts/` directory; all scripts listed in `scripts/README.md` are required:

```
git clone <this repo> claude-toolkit
cd claude-toolkit
```

All scripts are set executable in the repository; no `chmod` step is needed.

4.3 Run the installer

```
bash scripts/install.sh
```

What `install.sh` does, in order:

1. Verifies `rsync`, `jq`, `awk` are on `PATH`; warns if `pandoc` is missing.
2. Symlinks every `claude-*.sh` script into `~/.local/bin/` so they're invokable as plain commands (`claude-unify`, `claude-add-account`, ...).

3. Adds `export PATH="$HOME/.local/bin:$PATH"` to `~/.bashrc` and `~/.zshrc` if missing.
4. Creates the managed alias file at `~/.local/share/claude-multi-account/aliases.sh` and ensures it's sourced from `~/.bashrc` and `~/.zshrc`.
5. Migrates any pre-existing inline alias `claude<N>=...` lines from `~/.bashrc` / `~/.zshrc` into the managed alias file, commenting out the originals (and saving a `.preunify.<timestamp>` backup of the rc file first).
6. Auto-detects every `~/.claude-*` directory and runs `claude-unify.sh` to merge them into `~/.claude-shared/`.
7. Re-renders the PDF and HTML from this markdown via `claude-export-docs.sh`.

After it finishes, reload your shell:

```
exec $SHELL -l
claude-list-accounts
```

You should see every account dir, with `CREDS: yes` and a `LINKS: 6/6` (or higher) indicator showing the shared symlinks are in place.

4.4 Add a third (or fourth, ...) account

```
claude-add-account
# prompts:
#   Alias name      [claude3]:
#   Config directory [/home/you/.claude-claude3]:
```

If you accept both defaults, you get:

- `~/.claude-claude3/` created with the full set of shared-state symlinks already in place.
- `alias claude3="CLAUDE_CONFIG_DIR=$HOME/.claude-claude3 \ $CLAUDE_BIN"` appended to the managed alias file.
- A reminder to authenticate: `claude3 /login`.

Custom alias names work just as well:

```
claude-add-account --alias work --dir ~/.claude-work --yes
work /login
```

4.5 Verify the result

```
claude-list-accounts

# Expected output (your alias names may differ):
# ALIAS      CONFIG DIR
#          CREDS  LINKS  NOTES
# claude1    /home/you/.claude-milos85vasic
#          yes    6/6
```



```
# claude2      /home/you/.claude-milos85vasic2nd
              yes      6/6
# claude3      /home/you/.claude-claude3
              no      6/6
#
# Shared store: /home/you/.claude-shared
# Alias file:   /home/you/.local/share/claude-multi-account/
                aliases.sh
```

You can confirm symlinks resolve correctly:

```
readlink -f ~/.claude-milos85vasic/projects
# /home/you/.claude-shared/projects

readlink -f ~/.claude-milos85vasic2nd/projects
# /home/you/.claude-shared/projects

# Both should print the same path. The same is true for todos,
# plans,
# settings.json, history.jsonl, CLAUDE.md, etc.
```

4.6 Rollback if needed

Every directory or file the unifier touched was moved to `<path>.preunify.<timestamp>` before the symlink was created. To restore:

```
claude-rollback
# moves ~/.claude-shared aside and restores every .preunify.*
  backup
```

The shared store isn't deleted — it's renamed to `~/.claude-shared.removed.<timestamp>` so you can manually recover any data written through the symlinks before rolling back.

5. Operational notes

5.1 Concurrent use of two accounts

`sessions/` is shared in this configuration. A live Claude Code session holds its lockfile under `sessions/`, and two concurrent processes writing to the same lockfile path can confuse each other. **Avoid running `claude1` and `claude2` simultaneously in two terminals.** If you need true concurrent use, override the unifier to keep `sessions/` private:

```
# in claude-unify.sh, remove "sessions" from SHARED_ITEMS
# then run claude-unify again – it'll relink sessions/ as per-
  account
```

history.jsonl is append-only and is mostly safe under concurrent writes (worst case: a small number of duplicate lines, which the next unify run will de-dupe).

5.2 Plugin path rewriting

Claude Code records absolute paths in `plugins/installed_plugins.json` and `plugins/known_marketplaces.json`. These paths can point to one of:

- `~/.claude/plugins/cache/...` (user-scope installs from any account)
- `~/.claude-<name>/plugins/cache/...` (project-scope installs from that account)

The unifier rsyncs plugin code from all three roots (`~/.claude/plugins/`, plus every account's `plugins/`) into `~/.claude-shared/plugins/`, then rewrites every `installPath` / `installLocation` to point under `~/.claude-shared/plugins/`. It also symlinks `~/.claude/plugins/cache` and `~/.claude/plugins/marketplaces` into the shared store, so any future user-scope install lands in the unified spot.

If you ever install a plugin and Claude reports "plugin code not found," re-run `claude-unify.sh` to repath the manifest.

5.3 Adding a new account from scratch (no existing dir)

The `claude-add-account.sh` script works whether or not the target dir exists yet. If it doesn't exist, the script creates an empty dir, links every shared item into it, and registers the alias. The new account has no credentials of its own — your next step is `claudeN / login`.

If you want to also pre-populate the new account with a `.claude.json` template (e.g., to skip the new-user warmup tips), copy one from an existing account *after* the add-account script finishes:

```
cp ~/.claude-milos85vasic2nd/.claude.json ~/.claude-
  claude3/.claude.json
# but DO NOT copy .credentials.json — that would leak the source
# account's OAuth tokens to the new account
```

5.4 Removing an account

```
claude-remove-account --alias claude3
# default: archives ~/.claude-claude3 -> ~/.claude-
  claude3.removed.<timestamp>
# add --delete to remove it instead
```

The shared store is left intact; other accounts continue to function.

5.5 Backups created during install

The installer never deletes existing data. Every directory or file it replaced lives at `<original-path>.preunify.<timestamp>` (or `<original-path>.removed.<timestamp>` for rollback). You can safely delete those backups once you've confirmed the new setup works:

```
find ~ -maxdepth 3 -name '*.preunify.*' -print      # see what's
      there
find ~ -maxdepth 3 -name '*.preunify.*' -delete     # delete once
      happy
```

A pre-migration tarball is also placed in `~/Documents/claude-config-backups/` by the original migration. Keep it until you're confident in the new layout.

6. Cross-platform reproduction

Linux

Everything in this document was developed and tested on Linux 6.12 with bash 5.2, jq 1.8.1, rsync 3.2.7, pandoc 3.9. No Linux-specific behaviours are relied on; the scripts pass shellcheck clean and use only POSIX-portable tools plus jq.

macOS

macOS ships an older bash (3.2) by default. Install GNU bash via Homebrew (`brew install bash`) and update the shebangs only if you hit syntax issues — the scripts use bash 4+ features only where the fallback is explicit (mapfile, associative arrays). On a default macOS:

```
brew install bash jq rsync gawk pandoc weasyprint
echo "$(brew --prefix)/bin/bash" | sudo tee -a /etc/shells
chsh -s "$(brew --prefix)/bin/bash"
```

Everything else (paths, `$HOME`, `~/.claude-*` convention) is identical to Linux.

WSL / Windows Subsystem for Linux

WSL is just Linux for our purposes. Use the Linux instructions. If you also run Claude Code natively on Windows, the Windows version uses a different config root entirely (%APPDATA%\claude) and is not covered by this toolkit.

7. Tests

Everything in this toolkit is covered by a hermetic test suite under `~/Documents/scripts/tests/`. Each test runs against a temporary `$HOME` created by `mktemp -d -t cma-test.XXXX`; the real `~/claude-*` directories are never touched. After the test process exits, the sandbox is `rm -rf'd` (with a safety check that the path basename starts with `cma-test.`).

7.1 Running the suite

```
# All tests:
bash scripts/tests/run-all.sh

# A single file (no test_prefix, no .sh suffix):
bash scripts/tests/run-all.sh unify
bash scripts/tests/run-all.sh add_remove
```

Output is line-based:

```
==> test_unify.sh
```

```
• every shared item becomes a symlink into the shared store
[PASS] .claude-acct1 projects linked: ...
[PASS] .claude-acct2 projects linked: ...
• settings.json enabledPlugins is the union {a,b,c}
[PASS] union (.enabledPlugins | keys | sort | join(",") =
a,b,c)
...
```

```
✓ 33 passed, 0 failed
```

The orchestrator's exit code is 0 only if every `test_*.sh` file's own summary returned 0.

7.2 What each test covers

The suite has 10 test files, all auto-discovered by `run-all.sh`:

File	What it asserts
test_lib.sh	

File	What it asserts
	cma_validate_alias accepts/rejects correctly; cma_suggest_alias returns the next free claudeN (skipping past existing aliases, not just counting); cma_write_alias is idempotent; cma_remove_alias deletes; cma_ensure_alias_file patches .bashrc; cma_detect_accounts excludes the shared store. Every shared item becomes a symlink to \$SHARED_DIR; settings.json enabledPlugins is a strict union of every account's set; non-enabledPlugins top-level keys are preserved with right-most file winning; history.jsonl is concatenated and de- duplicated; on memory-file conflicts, the most recently touched account wins; plugins/installed_plugins.json and known_marketplaces.json have absolute paths rewritten to \$SHARED_DIR/ plugins/; .credentials.json and .claude.json are NOT symlinked; re-running claude- unify.sh produces a byte-identical settings.json (idempotency); N>2 accounts (adding acct3 and re-unifying) works; -- rollback restores every .preunify.* backup and moves the shared store aside. claude-add-account.sh --yes creates the dir with full symlink set and writes a shell-safe alias line; refuses to overwrite an existing dir; accepts a custom alias name + custom dir; validates alias names against the safe regex; claude-remove-account.sh --archive moves the dir aside and removes the alias; --delete actually deletes; unknown alias errors out.
test_unify.sh	
test_add_remove.sh	Empty host (no accounts) renders just the header; populated host renders one row per detected account dir; the shared store path is printed; accounts missing credentials show CREDs: no.
test_list.sh	
test_opencode.sh	Builds a fake Claude plugin cache in a throwaway \$HOME and asserts the generated OpenCode config has the right shape: skills wired, MCP servers translated (wrapped + bare formats), dedup, collision-rename, secret/runtime enable gating, \$ {CLAUDE_PLUGIN_ROOT} expansion, instruction inclusion, pre-existing config preserved, idempotency, and --dry-run safety. No

File	What it asserts
	network, no real ~/.claude, no real opencode binary required.
test_sessions.sh	Verifies sessions, project memory, and todos created under one account become physically visible to every other account after unify. Covers: .claude.json projects subtree union; auth-private keys (userID, oauthAccount, etc.) preserved per-account; session JSONL byte-identical from every account's symlinked dir; project memory cross-account visibility; cma_merge_claude_json idempotency; claude-sync-state.sh push/pull round-trip.
test_coverage.sh	Coverage for lib.sh behaviors not exercised in test_lib.sh (new in v1.7.8): cma_ensure_alias_file fresh creation + idempotency + old-format migration; cma_can_prompt under CMA_NONINTERACTIVE=1 and no-tty; cma_enable_plugins JSON shape and additive behaviour; cma_link_shared_items turning all CMA_SHARED_ITEMS into symlinks; stats-cache.json newest-by-mtime selection via run_unify.
test_providers.sh	Tests for the provider-alias feature (claude-providers). Primary focus: account-detection regression — provider dirs (~/.claude-prov-*) must be invisible to cma_detect_accounts so they never merge into real-account auth or disturb claude-unify / claude-add-account.
test_claude.sh	Proves claudeN aliases are untouched by provider code: cma_run includes claude-sync-state pull/push hooks, contains no proxy or transformer code; cma_run_provider has proxy detection. Reads the live alias file; SKIPS if no alias file is present.

7.3 How the test harness works

Two small libraries sit under tests/lib/:

- **assert.sh** — it, assert_eq, assert_file, assert_dir, assert_symlink_to, assert_not_symlink, assert_file_contains, assert_file_not_contains, assert_jq, assert_lines, assert_exit. Each helper prints a [PASS] or [FAIL] line and adjusts the counters used by summary at end-of-file.
- **sandbox.sh** — make_sandbox (creates a tempdir and overrides HOME, SHARED_DIR, ALIAS_FILE, DEFAULT_DIR, CLAUDE_BIN),

```
make_account NAME [--plugins] [--settings JSON] [--history a|b|
c] [--memory KEY:VALUE] [--todo SUBJECT] (builds a realistic per-
account fixture dir), plus thin wrappers run_unify,
run_add_account, etc.
```

One subtlety worth knowing: `lib.sh` (the production helper) has `set -euo pipefail`. When a test sources it for unit-testing the helper functions, the `set -e` flag stays active in the test shell. The unit test file (`test_lib.sh`) therefore runs `set +e` immediately after sourcing `lib.sh` so that intentional failures (e.g. `cma_validate_alias "bad name"`) don't abort the whole test process. End-to-end tests don't have this problem because they invoke the scripts as external processes (via `run_unify`, etc.), which run in their own shell.

7.4 Adding a new test

1. Create `tests/test_<feature>.sh`. Mark it executable.
2. Source the two libs:

```
TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"
```

3. Call `make_sandbox` once at the top.
 4. Use it `"..."` to label each scenario, then a sequence of `assert_*` calls.
 5. End the file with summary so the orchestrator picks up the result.
 6. Re-run `bash tests/run-all.sh` — auto-discovery will pick it up.
-

8. The complete script source

For convenience and audit, the full source of the toolkit is reproduced below. The authoritative copies live at `~/Documents/scripts/`.

8.1 `lib.sh`

```
#!/usr/bin/env bash
# lib.sh – shared functions for the Claude multi-account toolkit.
# Sourced by the other scripts; no side effects on its own.

set -euo pipefail

# Which rc files to source the alias file from. macOS interactive
# shell is
```

```

# zsh; touching .bashrc there just creates noise the user has to
# clean up.
# On Linux we keep both because either may be the login shell.
if [[ "$(uname -s)" == "Darwin" ]]; then
    CMA_RC_FILES=("$HOME/.zshrc")
else
    CMA_RC_FILES=("$HOME/.bashrc" "$HOME/.zshrc")
fi

# Resolve paths the user can override. SHARED_DIR is the single
# canonical
# location for cross-account state; ALIAS_FILE is the rc-sourced
# file we
# manage aliases through; ACCOUNT_PREFIX is the dir-name prefix
# for new
# per-account config directories.
: "${SHARED_DIR:= $HOME/.claude-shared}"
: "${ALIAS_FILE:= $HOME/.local/share/claude-multi-account/
aliases.sh}"
: "${ACCOUNT_PREFIX:= .claude-}"

# Resolve the Claude Code binary for the alias wrappers. Prefer an
# explicit
# CLAUDE_BIN, then $PATH, then the common install locations. npm's
# global prefix
# varies per host (e.g. ~/.npm-global vs ~/.local vs Homebrew), so
# a fixed
# ~/.local/bin default mis-points on hosts where `npm i -g
# @anthropic-ai/...`
# landed elsewhere – making EVERY alias launch fail "No such
# file". Checking
# the real locations keeps a fresh install working without a
# manual symlink.
cma_resolve_claude_bin() {
    if [ -n "${CLAUDE_BIN:-}" ]; then printf '%s\n' "$CLAUDE_BIN";
    return 0; fi
    local c; if c="$(command -v claude 2>/dev/null)"; then printf
    '%s\n' "$c"; return 0; fi
    local p
    for p in "$HOME/.local/bin/claude" "$HOME/.npm-global/bin/
claude" \
/opt/homebrew/bin/claude /usr/local/bin/claude; do
        [ -x "$p" ] && { printf '%s\n' "$p"; return 0; }
    done
    printf '%s\n' "$HOME/.local/bin/claude"
    # fallback (created by install/symlink)
}
CLAUDE_BIN_DEFAULT="$(cma_resolve_claude_bin)"

cma_log() { printf '\033[36m[ cma ]\033[0m %s\n' "$*" >&2; }
cma_warn() { printf '\033[33m[ cma warn ]\033[0m %s\n' "$*" >&2; }
cma_err() { printf '\033[31m[ cma err ]\033[0m %s\n' "$*" >&2; }
cma_die() { cma_err "$*"; exit 1; }

```



```

cma_require() {
    command -v "$1" >/dev/null 2>&1 || cma_die "missing required
        tool: $1"
}

# Keys inside .claude.json that must NEVER leak across accounts
# (auth + identity).
# Everything else in .claude.json – projects map, UX state,
# caches, etc. – is
# safely shareable. New auth keys should be added here, not
# assumed shareable.
CMA_CLAUDE_JSON_PRIVATE_KEYS='["userID","oauthAccount","firstStartTime","claudeCoo

# Deep-merge every account's .claude.json so each account's file
# ends up with
# its OWN auth keys + the UNION of all other top-level keys
# (rightmost-wins
# for scalar conflicts, recursive object merge for `projects` and
# friends).
#
# Args: one or more account config dirs.
# Side effect: rewrites each $acct/.claude.json in place. Skips an
# account
# that doesn't yet have a .claude.json (e.g. brand-new dir before
# first login).
cma_merge_claude_json() {
    local accts=("$@")
    (( ${#accts[@]} >= 1 )) || return 0
    cma_require jq

    # Build the merged "shared portion" (everything except private
    # keys).
    local shared_tmp; shared_tmp="$(mktemp "${TMPDIR:-/tmp}/
        cma.XXXXXX")"
    printf '{}\n' > "$shared_tmp"
    local acct prev="$shared_tmp"
    for acct in "${accts[@]}; do
        local f="$acct/.claude.json"
        [[ -s "$f" ]] || continue
        local next; next="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
        # $a (accumulator) * ($b stripped of private keys). jq's `*`
        # is recursive
        # deep-merge for objects; rightmost wins on scalar conflicts
        # AND on array
        # values (arrays are replaced, not element-unioned – e.g. a
        # per-project

        # prompt-history array takes the last account's copy).
        # This is a deliberate

        # deep-merge trade-off: the projects subtree is unioned at
        # the object-key
        # level (which is what the cross-account session/MCP/memory
        # index needs);

```

```

# blind element-level array union would be wrong for config-
# style arrays.
if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
  . as [$a, $b]
  | ($b | with_entries(select(.key as $k | $priv | index($k) |
    not))) as $shared_b
  | ($a // {}) * ($shared_b // {})
' "$prev" "$f" > "$next"; then
  rm -f "$next"
  cma_warn ".claude.json in
    $acct is not valid JSON – skipping its contribution"
  continue
fi
rm -f "$prev"
prev="$next"
done

# Sticky-true TRUST preservation (§11.4 anti-bluff config
# hygiene): the jq `*`
# merge above is last-writer-wins on scalars, so a per-project
# `projects[<path>].hasTrustDialogAccepted` bit trusted under
# one alias could be
# DEMOTED true->false by a later account that lacks it – which
# made Claude Code's
# per-workspace trust dialog ("read, edit, and execute files
# here") reappear on
# every provider alias. Fix: OR the trust bit across all
# accounts – once a
# project path is trusted anywhere, it stays trusted in the
# merged portion.
local _tr_files=() _tr_acct
for _tr_acct in "${accts[@]}"; do [[ -s
  "$_tr_acct/.claude.json" ]] &&
  _tr_files+=("$_tr_acct/.claude.json"); done
if (( ${#_tr_files[@]} >= 1 )); then
  local _trusted
  _trusted="$(jq -s '[ .[] | (.projects // {}) | to_entries[]
    | select(.value.hasTrustDialogAccepted ==
      true) | .key ] | unique' \
    "${_tr_files[@]}" 2>/dev/null || printf '[]')"
  if [[ -n "$_trusted" && "$_trusted" != "[]" ]]; then
    local _tt; _tt="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if jq --argjson trusted "$_trusted" '
      .projects // = {}
      | reduce ($trusted[]) as $p
      (.; .projects[$p].hasTrustDialogAccepted = true)
      ' "$prev" > "$_tt" 2>/dev/null && jq -e . "$_tt" >/dev/
      null 2>&1; then
      mv "$_tt" "$prev"
    else rm -f "$_tt"; fi
  fi
fi

```

```

# Now $prev holds the merged shared portion. Write each
# account's file with
# its own private keys overlaid on top of the shared portion.
for acct in "${accts[@]}; do
    local f="$acct/.claude.json" out
    out="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if [[ -s "$f" ]]; then
        # Guard against set -e: a corrupt $f makes jq fail, which
        # would abort
        # the whole loop and leave the remaining accounts unsynced.
        if ! jq -s --argjson priv "$CMA_CLAUDE_JSON_PRIVATE_KEYS" '
            . as [$shared, $own]
            | ($own | with_entries(select(.key as $k | $priv |
                index($k)))) as $priv_only
            | ($shared // {}) * $priv_only
        ' "$prev" "$f" > "$out" 2>/dev/null; then
            rm -f "$out"
            cma_warn ".claude.json for $acct could not be merged
                (invalid JSON?) – leaving original alone"
            continue
        fi
    else
        # New account, no existing .claude.json. Seed with shared
        # content only.
        cp "$prev" "$out"
    fi
    # Final sanity: only replace if the output is parseable JSON.
    if jq -e . "$out" >/dev/null 2>&1; then
        mv "$out" "$f"
    else
        rm -f "$out"
        cma_warn "merged .claude.json for $acct was invalid –
            leaving original file untouched"
    fi
done
rm -f "$prev"
}

# Detect Linux vs macOS for platform-specific commands.
cma_os() {
    case "$(uname -s)" in
        Linux*)    echo linux ;;
        Darwin*)  echo macos ;;
        *)         echo unknown ;;
    esac
}

# Portable realpath. BSD/macOS `readlink` has no `-f`, so
# `readlink -f` is a hard
# error there (prints "illegal option -- f", returns empty) even
# after a bash-4
# re-exec, because /usr/bin/readlink stays BSD. This resolves a
# path to its

```

```

# canonical absolute form by walking the symlink chain with
# single-arg
# `readlink` (supported everywhere) plus `pwd -P` – the same
# technique every
# script's LIB_DIR resolver uses. Safe under `set -e`.
cma_realpath() {
    local p="$1" t dir base
    while [ -L "$p" ]; do
        t="$(readlink "$p")"
        case "$t" in /*) p="$t" ;; *) p="$(dirname "$p")/$t" ;; esac
    done
    dir="$(cd "$(dirname "$p")" 2>/dev/null && pwd -P)" || dir="$(
        (dirname "$p")"
    base="$(basename "$p")"
    printf '%s/%s\n' "$dir" "$base"
}

# Find all existing Claude account config directories under $HOME,
# matching
# the convention `.claude-<name>`. Echoes absolute paths, one per
# line,
# sorted. The default `~/.claude` is intentionally excluded
# because we treat
# it as the shared user-scope spot, not an account dir.
cma_detect_accounts() {
    local d
    while IFS= read -r d; do
        [[ "$d" == *"-shared" ]] && continue
        # Provider-alias dirs (~/.claude-prov-<id>, created by claude-
        # providers)
        # are account-like for SHARED state but must NEVER be merged
        # into
        # real-account auth/identity or unify, so they're excluded
        # from detection
        # exactly like *-shared. This is the linchpin that keeps the
        # existing
        # claudeN accounts and add-account untouched by the provider
        # feature.
        [[ "$(basename "$d")" == "${ACCOUNT_PREFIX}prov-*" ]] &&
        continue

        # The claude-code-router config dir and any *.lock dir are
        # not accounts.
        [[ "$(basename "$d")" == "${ACCOUNT_PREFIX}code-router" ]] &&
        continue
        [[ "$(basename "$d")" == *.lock ]] && continue
        # Empty dirs always count (a brand-new account before any
        # claude run).
        if [[ -z "$(ls -A "$d" 2>/dev/null)" ]]; then
            echo "$d"
            continue
        fi
        # Non-empty: must look like a Claude account (at least one
        # tell-tale

```

```

# file/dir). Filters out dirs that merely match the `.claude-
*` prefix
# but belong to other tools (e.g. `.claude-server-commander`).
if [[ -d "$d/projects" || -d "$d/todos" || -d "$d/plugins" \
    || -f "$d/.claude.json" || -f "$d/.credentials.json" \
    || -f "$d/history.jsonl" ]]; then
    echo "$d"
fi
done < <(find "$HOME" -maxdepth 1 -type d -name "$
{ACCOUNT_PREFIX}*" 2>/dev/null | sort)
}

# Read all aliases of the form `alias name=...` from $ALIAS_FILE
and print
# their names (one per line). Returns nothing if the file is
absent.
cma_existing_aliases() {
    [[ -f "$ALIAS_FILE" ]] || return 0
    awk -F'[ =]+' '/^[[[:space:]]*alias[[[:space:]]+/{print $2}'
        "$ALIAS_FILE"
}

# Suggest the next free `claude<N>` alias by scanning current
aliases.
# E.g., if claude1 and claude2 exist, returns "claude3".
cma_suggest_alias() {
    local highest=0 n
    for a in $(cma_existing_aliases); do
        if [[ "$a" =~ ^claude([0-9]+)$ ]]; then
            n="${BASH_REMATCH[1]}"
            (( n > highest )) && highest="$n"
        fi
    done
    echo "claude$((highest + 1))"
}

# Validate that an alias name is safe to embed in a generated
`alias ...=`
# line – strictly alphanumeric + underscore/hyphen, starting with
a letter.
cma_validate_alias() {
    [[ "$1" =~ ^[a-zA-Z][a-zA-Z0-9_-]*$ ]] || cma_die
        "invalid alias name: $1"
}

# Match an rc-file line that sources an aliases.sh, capturing the
path:
# BASH_REMATCH[2] = the (possibly $HOME/~/*quoted) target path.
# Anchored so leading-# comment lines (e.g. "# migrated to .../
aliases.sh: ...")
# never match. Used by the prune + dedup helpers below.
CMA_ALIAS_SRC_RE='^[[[:space:]]*(source|\.)[[[:space:]]+]"
([^^[:space:]]*aliases\.sh)"?[[[:space:]]]*$'

```

```

# Remove rc-file lines that source an aliases.sh whose target no
# longer exists
# (a stale path from a moved install, or a transient ALIAS_FILE
# used in testing).
# Without this, a deleted alias file leaves a dangling line that
# errors
# "-bash: ../aliases.sh: No such file or directory" on every new
# login shell.
cma_prune_stale_alias_sources() {
    local rc="$1" tmp line target changed=0
    [[ -f "$rc" ]] || return 0
    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")" || return 0
    while IFS= read -r line || [[ -n "$line" ]]; do
        if [[ "$line" =~ $CMA_ALIAS_SRC_RE ]]; then
            target="${BASH_REMATCH[2]}"; target="${target/#\$HOME/
            \$HOME}"; target="${target/#\~/\$HOME}"
            if [[ ! -f "$target" ]]; then changed=1; continue; fi
        fi
        printf '%s\n' "$line" >> "$tmp"
    done < "$rc"
    if (( changed )); then mv "$tmp" "$rc"; cma_log "pruned stale
        aliases.sh source line(s) from $rc"; else rm -f "$tmp"; fi
}

# True if $rc already sources a file resolving to $2 (across `./
# `source` and
# $HOME/~ /absolute forms), so we never append a duplicate source
# line.
cma_rc_sources_alias_file() {
    local rc="$1" want="$2" line target want_real
    [[ -f "$rc" ]] || return 1
    want_real="$(cma_realpath "$want" 2>/dev/null || printf '%s'
        "$want")"
    while IFS= read -r line || [[ -n "$line" ]]; do
        if [[ "$line" =~ $CMA_ALIAS_SRC_RE ]]; then
            target="${BASH_REMATCH[2]}"; target="${target/#\$HOME/
            \$HOME}"; target="${target/#\~/\$HOME}"
            [[ "$(cma_realpath "$target" 2>/dev/null || printf '%s'
                "$target")" == "$want_real" ]] && return 0
        fi
    done < "$rc"
    return 1
}

# Ensure $ALIAS_FILE exists with the header sentinel and is
# sourced from
# the user's rc files. Idempotent – safe to call repeatedly.
cma_ensure_alias_file() {
    mkdir -p "$(dirname "$ALIAS_FILE")"
    if [[ ! -f "$ALIAS_FILE" ]]; then
        cat > "$ALIAS_FILE" <<EOF
# Managed by claude-multi-account. Do not edit by hand; use
# ~/.local/bin/claude-add-account to add accounts.
export CLAUDE_BIN="${CLAUDE_BIN_DEFAULT}"

```

EOF

```
fi
# Migration: rewrite any pre-wrapper alias lines that invoke
# $CLAUDE_BIN
# directly so they go through cma_run instead. Pre-existing
# installs need
# this on the first re-run of install.sh after the runtime-sync
# feature
# landed; idempotent (a line already using cma_run is left
# alone).
# shellcheck disable=SC2016 # $CLAUDE_BIN is a literal in the
# regex/sed pattern, not a shell expansion
if grep -qE '^alias[[:space:]]+[[:space:]]+=.*CLAUDE_CONFIG_DIR=[^ ]+
[[:space:]]+\|?\$CLAUDE_BIN"$' "$ALIAS_FILE"; then
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"

    # shellcheck disable=SC2016 # $CLAUDE_BIN is a literal in
    # the sed pattern, matching alias file content
    sed -E 's|(^alias[[:space:]]+[[:space:]]+=)"(CLAUDE_CONFIG_DIR=[^ ]+
[[:space:]]+\|?\$CLAUDE_BIN"$|\1"\2 cma_run"|'
"$ALIAS_FILE" > "$tmp"
    mv "$tmp" "$ALIAS_FILE"
    cma_log "migrated existing aliases in $ALIAS_FILE to use
    cma_run wrapper"
fi

# Migration: an existing alias file may carry a stale
# CLAUDE_BIN pointing at a
# path that does not exist on THIS host (e.g. ~/.local/bin/
# claude when npm put
# claude in ~/.npm-global/bin – the amber.local case). If the
# recorded
# CLAUDE_BIN is not executable, rewrite it to a resolved one so
# every alias
# launch finds claude without a manual symlink.
local _cur_cb _cur_cb_exp _new_cb
# `|| _cur_cb=""` is LOAD-BEARING: under `set -euo pipefail` a
# no-match grep
# (an older/hand-edited alias file with no `export CLAUDE_BIN=`
# line) would
# abort cma_ensure_alias_file mid-run.
_cur_cb="$(grep -m1 '^export CLAUDE_BIN=' "$ALIAS_FILE" 2>/dev/
null)" || _cur_cb=""
_cur_cb="${_cur_cb#export CLAUDE_BIN=}"; _cur_cb="${
{_cur_cb#\}}""; _cur_cb="${_cur_cb%\"}"
_cur_cb_exp="${_cur_cb/#\${HOME}/${HOME}}"; _cur_cb_exp="${
{_cur_cb_exp/#\~/\${HOME}}}"
if [[ -n "$_cur_cb" && ! -x "$_cur_cb_exp" ]]; then
    _new_cb="$(cma_resolve_claude_bin)"
    if [[ "$_new_cb" != "$_cur_cb" && -x "${_new_cb/#\${HOME}/
${HOME}}" ]]; then
        local tmp_cb; tmp_cb="$(mktemp "${TMPDIR:-/tmp}/
cma.XXXXXX")"
        sed 's|^export CLAUDE_BIN=.*|export
        CLAUDE_BIN=\$_new_cb\|"' "$ALIAS_FILE" > "$tmp_cb"
        mv "$tmp_cb" "$ALIAS_FILE"
```

```

    cma_log "migrated stale CLAUDE_BIN -> $_new_cb"
fi
fi
# Migration: regenerate an outdated cma_run that lacks the
# provider-env

    # isolation guard (the 'unset ANTHROPIC_' marker). Without
    # it, a native
# claudeN launched in a shell that previously ran a provider
# alias would
# INHERIT that provider's exported ANTHROPIC_BASE_URL/
# AUTH_TOKEN/MODEL and
# talk to the wrong API (e.g. claude1 hitting xiaomi's
# endpoint). Drop only
# the function block; the correct version is re-appended below.
# NOTE: match with LITERAL parens `^cma_run()` – NOT `^cma_run\
# (\)` . In both
# grep BRE and awk ERE, `\\(\\)` is an *empty capture group* that
# matches the

    # empty string, so `^cma_run\\(\\)` matches any line
    # starting with "cma_run",
# INCLUDING "cma_run_provider()". That false match made the re-
# append guard
# below think cma_run still existed after it was stripped,
# dropping the
# function entirely. Literal `()` matches only the real
# cma_run() header.
# Regenerate cma_run if its body is missing ANY current marker:
# * 'unset ANTHROPIC_' – provider-env isolation (native must
#   not inherit a
#   provider endpoint left exported in the shell),
# * 'claude-session' – the per-project auto-session naming
#   integration, and
# * 'claude-cwd-hook' – the optional project-agnostic pre-
#   launch working-dir
# hook (lets a consuming project bind each alias to its own
# checkout).
# A stale wrapper lacking ANY would silently misbehave (wrong
# endpoint,
# unnamed sessions, or no per-alias cwd) and must self-heal on
# the next

    # install/ensure. The earlier bug checked only the first
    # marker, so wrappers
# predating auto-session never regained it.
local _cma_run_body
_cma_run_body="$(awk '/^cma_run\\(\\) ?\\/{f=1} f{print} f&&/^}/
{exit}' "$ALIAS_FILE" 2>/dev/null)"
if grep -q '^cma_run()' "$ALIAS_FILE" \
&& { ! printf '%s\n' "$_cma_run_body" | grep -q 'unset
ANTHROPIC_' \
|| ! printf '%s\n' "$_cma_run_body" | grep -q 'claude-
session' \
|| ! printf '%s\n' "$_cma_run_body" | grep -q 'claude-
cwd-hook' \

```



```

        || ! printf '%s\n' "$_cma_run_body" | grep -q 'apply-
        color'; }; then
local tmp_run; tmp_run="$(mktemp "${TMPDIR:-/tmp}/
        cma.XXXXXX")"
awk '
        /^cma_run\( \) ?\{/ { skip=1 }
        skip && /^}/ { skip=0; next }
        !skip { print }
' "$ALIAS_FILE" > "$tmp_run"
mv "$tmp_run" "$ALIAS_FILE"
cma_log "migrated outdated cma_run (provider-env isolation +
        auto-session)"
fi
# Ensure the cma_run wrapper is present in the alias file. This
        is the
# runtime hook that keeps .claude.json projects/session state
        synchronized
# across every account: pull merged state before launch, push
        back after exit.
if ! grep -q '^cma_run()' "$ALIAS_FILE"; then
        cat >> "$ALIAS_FILE" <<'EOF'

# Wrapper: keeps .claude.json projects/session index synced across
        every
# logged-in account. Pulls merged state from every account into
        the launching
# one before claude runs; pushes the post-session state back out
        after exit.
# Cheap (jq deep-merge of one ~50KB file per account), runs
        unconditionally.
cma_run() {
        # Provider-env isolation: native claudeN must talk to the real
        Anthropic API.
        # A provider alias run earlier in THIS shell exports
        ANTHROPIC_BASE_URL etc.;
        # those persist and would otherwise leak into this native launch
        (claude1
        # silently using a provider's endpoint). Clear them so native is
        always clean.
        unset ANTHROPIC_BASE_URL ANTHROPIC_AUTH_TOKEN ANTHROPIC_MODEL
        ANTHROPIC_SMALL_FAST_MODEL
        # Optional project-agnostic working-dir hook (opt-in; no-op when
        absent). A
        # consuming project can bind each alias to its own checkout
        (e.g. one git
        # worktree per track) so parallel aliases don't contend on a
        single shared
        # tree. The toolkit knows NOTHING about what the hook resolves –
        it only cd's
        # into a real directory the hook prints on stdout (nothing
        printed => stay put).
        # Runs before claude-session below so the auto-session keys to
        the worktree
        # root. Escape hatch: MULTITRACK_DISABLE=1 (honored inside the
        hook itself).

```

```

local _cma_cwd_hook="${CMA_CWD_HOOK:-$HOME/.local/bin/claude-
    cwd-hook}" _cma_cwd_label _cma_cwd_target
if [[ -x "$_cma_cwd_hook" ]]; then
    _cma_cwd_label="$(basename "${CLAUDE_CONFIG_DIR:-claude}")";
    _cma_cwd_label="${_cma_cwd_label#.claude-}"
    _cma_cwd_target="$($_cma_cwd_hook "$_cma_cwd_label" 2>/dev/
        null || true)"
    if [[ -n "$_cma_cwd_target" && -d "$_cma_cwd_target" ]]; then
        cd "$_cma_cwd_target" 2>/dev/null || true; fi
fi
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
# Auto session-per-project: when launched with NO args, resume
# (or create) the
# one long-lived session for this project root, name it after
# the root dir,
# trust the workspace, and hint the alias color. Only when bare
# so explicit
# user flags (-p, --resume, a prompt, ...) are always respected
# verbatim.
# claude-session emits only "--resume <uuid>" or "--session-id
# <uuid> --name
# <kebab>" (no shell metacharacters), so eval-splitting is safe
# and works in
# both bash and zsh (zsh does not word-split unquoted
# expansions).
local _cma_label=""
if [[ $# -eq 0 && -x "$HOME/.local/bin/claude-session" ]]; then
    local _cma_sf
    _cma_sf="$("$HOME/.local/bin/claude-session" flags
        "$CLAUDE_CONFIG_DIR" 2>/dev/null || true)"
    _cma_label="$(basename "${CLAUDE_CONFIG_DIR:-claude}")";
    _cma_label="${_cma_label#.claude-}"
    "$HOME/.local/bin/claude-session" hint "$_cma_label" 2>/dev/
        null || true
    eval "set -- $_cma_sf"
    # Auto-apply the per-alias color: a resumable session's jsonl
    # exists now, so
    # colour it before launch; a brand-new session's file appears
    # during launch,
    # so we colour it again after exit (see post-launch call
    # below).
    "$HOME/.local/bin/claude-session" apply-color
        "$CLAUDE_CONFIG_DIR" "$_cma_label" 2>/dev/null || true
fi
"$CLAUDE_BIN" "$@"
local rc=$?
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
[[ -n "$_cma_label" && -x "$HOME/.local/bin/claude-session" ]]
&& \

```

```

"$HOME/.local/bin/claude-session" apply-color
"$CLAUDE_CONFIG_DIR" "$_cma_label" 2>/dev/null || true
return $rc
}
EOF
fi
# Ensure the cma_run_provider wrapper is present. This launches
# Claude Code
# against a non-Anthropic provider: it reads the per-provider
# non-secret env
# file, injects the API key from the keys file at launch (the
# toolkit never
# persists secrets itself), then runs claude directly (native
# transport) or

# via claude-code-router (router transport). Self-
# contained: the user's shell
# sources only this alias file, not lib.sh.
#
# Migration: if cma_run_provider exists but its body is missing
# the
# sync-state call, it's an outdated version that breaks cross-
# provider
# /resume. Remove ONLY the function block (keeping any alias
# lines that
# follow it) so the correct version gets re-appended below.
#
# The detection MUST be scoped to the function body and match
# the real
# on-disk text. Two prior bugs lived here:
# 1. cma_run also contains a sync-state call, so a whole-file
# grep can
# never tell an outdated provider wrapper from a current
# one.

# 2. The emitted text is `.../claude-sync-state" pull` – a
# quote precedes
# the space – so grepping for "claude-sync-state pull"
# (with a space)
# never matched, making the migration mis-fire on EVERY
# alias write.
# That chopped previously-written aliases (and claudeN
# aliases that
# come after the functions), corrupting the alias file.
# We now extract the function body (cma_run_provider() .. its
# closing
# brace) and match the bare command name, which is quote/space
# agnostic.
# Regenerate when the installed function predates ANY of these:
# the
# cross-provider sync-state calls ('claude-sync-state'), the
# nounset-safe
# keys sourcing ('set -a +u'), the per-project auto-session
# integration
# ('claude-session'), the input-context token-limit guard

```

```

# ('CLAUDE_CODE_AUTO_COMPACT_WINDOW'), or the SHARED_DIR-based
# proxy resolution
# ('_cma_proxy_dir', replacing a broken $LIB_DIR that disabled
# all proxies).
# Each marker lives only in the current heredoc, so once
# regenerated the
# function stops re-triggering.
if grep -q '^cma_run_provider()' "$ALIAS_FILE"; then
    local _prov_body
    _prov_body="$(awk '/^cma_run_provider\(\) ?\{/{f=1} f{print}
    f&&/^}/{exit}' "$ALIAS_FILE")"
    # shellcheck disable=SC2016 # '>| "$tmp"' is a literal code
    # marker grepped for, not a var to expand
    if ! printf '%s\n' "$_prov_body" | grep -q 'claude-sync-
    state' || \
        ! printf '%s\n' "$_prov_body" | grep -q 'set -a +u' || \
        ! printf '%s\n' "$_prov_body" | grep -q 'claude-session'
        || \
        ! printf '%s\n' "$_prov_body" | grep -q 'apply-color' || \
        ! printf '%s\n' "$_prov_body" | grep -q
        'CLAUDE_CODE_AUTO_COMPACT_WINDOW' || \
        ! printf '%s\n' "$_prov_body" | grep -q '_cma_proxy_dir'
        || \
        ! printf '%s\n' "$_prov_body" | grep -qF 'command -v
        cma_log' || \
        ! printf '%s\n' "$_prov_body" | grep -qF '_cma_force' || \
        ! printf '%s\n' "$_prov_body" | grep -qF '>| "$tmp"'; then
        local tmp_prov; tmp_prov="$(mktemp "${TMPDIR:-/tmp}/
        cma.XXXXXX")"
        # Drop only the function block; preserve everything before
        # and after it.
        awk '
            /^cma_run_provider\(\) ?\{/{ skip=1 }
            skip && /^}/{ skip=0; next }
            !skip { print }
        ' "$ALIAS_FILE" >| "$tmp_prov"
        mv "$tmp_prov" "$ALIAS_FILE"
        cma_log "migrated outdated cma_run_provider (sync-state +
        nounset keys + noclobber-safe >| write + auto-compact-
        window + activation-gate)"
    fi
fi
if ! grep -q '^cma_run_provider()' "$ALIAS_FILE"; then
    cat >> "$ALIAS_FILE" <<'EOF'

cma_run_provider() {
    # --force bypasses the activation gate (operator override).
    # Accepted BOTH as
    # the very first arg (direct call: cma_run_provider --force
    # <id>) and as the
    # first arg after the id (alias path: `<alias> --force` expands
    # to
    # cma_run_provider <id> --force). Either way it is consumed, not
    # forwarded.
    local _cma_force=0

```

```

if [[ "${1:-}" == "--force" ]]; then _cma_force=1; shift; fi
local id="$1"; shift 2>/dev/null || true
if [[ "${1:-}" == "--force" ]]; then _cma_force=1; shift; fi
local pdir="$HOME/.local/share/claude-multi-account/providers"
local envf="$pdir/$id.env"
if [[ ! -f "$envf" ]]; then
    printf 'claude-providers: unknown provider %s (missing %s)\n'
        "$id" "$envf" >&2
    return 1
fi
# shellcheck disable=SC1090
source "$envf"
# Activation gate: only a 'verified' alias launches Claude Code.
#   A non-verified
# alias (unverified / failed / pending) prints a clear,
#   actionable message and
# refuses to launch, so a broken provider never surfaces as a
#   confusing
# in-session error. Status comes from the non-secret status
#   cache; this body is
# self-contained (no cma_* helpers) so it reads status.json with
#   jq inline.
if (( ! _cma_force )); then
    local _cma_sf="$pdir/status.json" _cma_st="pending"
    if command -v jq >/dev/null 2>&1 && [[ -s "$_cma_sf" ]]; then
        _cma_st="$(jq -r --arg i "$CMA_PROVIDER_ID" '.[$i].status //
            "pending"' "$_cma_sf" 2>/dev/null)"
        [[ -n "$_cma_st" && "$_cma_st" != "null" ]] ||
            _cma_st="pending"
    fi
    if [[ "$_cma_st" != "verified" ]]; then
        printf 'claude-providers: alias %s is %s – not launching.\n'
            "$CMA_PROVIDER_ID" "$_cma_st" >&2
        printf '  Re-verify: claude-providers sync    (re-runs
            verification for %s)\n' "$CMA_PROVIDER_ID" >&2
        printf '  Override (operator): run the alias with --force\n'
            >&2
        return 3
    fi
fi
local keysf="${CMA_KEYS_FILE:-$HOME/api_keys.sh}"
# Disable nounset while sourcing the user-controlled keys file:
#   it may have
# dangling refs (e.g. `export X=$UNSET`) that would abort the
#   source under a
# caller's `set -u`, leaving the token empty. Save/restore so we
#   never change
# the user's interactive shell options.
if [[ -f "$keysf" ]]; then
    case $- in *u*) local _cma_had_u=1 ;; *) local _cma_had_u=0 ;;
    esac
    set -a +u; source "$keysf"; set +a
    (( _cma_had_u )) && set -u
fi

```

```

# Indirect-expand the key var name. ${!var} is bash-only and a
# fatal error in
# zsh (the default macOS interactive shell this alias file is
# sourced into),
# so use eval, which works in both. CMA_PROVIDER_KEYVAR is a
# validated env
# var name ([A-Za-z_][A-Za-z0-9_]*), so this eval is safe.
local token="" _cma_xt=""
# Suppress xtrace around the indirect key read so an active `set
# -x` in the
# user's shell can't echo the secret to the terminal or a
# redirected log.
case $- in *x*) _cma_xt=1; set +x ;; esac
eval "token=\"\${$CMA_PROVIDER_KEYVAR:-}\"\"
[[ -n "$_cma_xt" ]] && set -x
if [[ -z "$token" ]]; then
    printf 'claude-providers: $%s is empty (set it in %s)\n'
        "$CMA_PROVIDER_KEYVAR" "$keysfn" >&2
    return 1
fi
export CLAUDE_CONFIG_DIR="$CMA_PROVIDER_CONFIG_DIR"
# Input-context guard (fixes "400 exceeded model token limit:
# 262144
# (requested: 311786)": tell Claude Code this provider's REAL
# context window
# so it auto-compacts (at window-13000) before a request
# overshoots the
# provider's hard input limit. Without this, Claude Code assumes
# Anthropic's
# own large (~1M) window and lets the prompt grow past a smaller
# provider's
# cap. Fully dynamic – the value is CMA_PROVIDER_CONTEXT_LIMIT,
# resolved from
# the models.dev catalog (limit.context) per selected model.
# Applies to BOTH
# transports (native + router), so every provider alias is
# protected.
# NOTE: this caps INPUT context; CLAUDE_CODE_MAX_OUTPUT_TOKENS
# (set below on
# the native path) caps OUTPUT – the two are independent halves
# of the guard.
[[ -n "${CMA_PROVIDER_CONTEXT_LIMIT:-}" ]] && \
    export
        CLAUDE_CODE_AUTO_COMPACT_WINDOW="$CMA_PROVIDER_CONTEXT_LIMIT"
# Sync .claude.json projects/session index across ALL accounts
# and providers
# so sessions created under any alias are visible from every
# other alias.
# Pull merged state before launch; push post-session state after
# exit.
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" pull "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
local rc

```

```

local _proxy_pid=""
if [[ "${CMA_PROVIDER_TRANSPORT:-native}" == "router" ]]; then
    if ! command -v ccr >/dev/null 2>&1; then
        printf 'claude-providers: provider %s needs claude-code-router.\n Install: npm install -g @musistudio/claude-code-router\n' "$id" >&2
        return 127
    fi
    # Upsert THIS provider into ccr config with the live key
    # (regenerated each
    # launch, chmod 600 – never stored by the toolkit), set it as
    # the active
    # route, then launch through ccr.
    local cfg="$HOME/.claude-code-router/config.json"
    base="$CMA_PROVIDER_BASE_URL"
    # Create the dir + config with restrictive perms from the
    # start: this file
    # will hold the live API key, so it must never be group/world
    # readable,
    # even transiently or if a later jq rewrite fails.
    ( umask 077; mkdir -p "$HOME/.claude-code-router"
      [[ -f "$cfg" ]] || echo '{"Providers":[],"Router":{}}' >
        "$cfg" )
    chmod 600 "$cfg" 2>/dev/null || true
    case "$base" in
        */chat/completions|*/v1beta/models|*/v1beta/models) ;;
        *) base="${base%}/chat/completions" ;;
    esac
    # Start compatibility proxy if the provider needs one (e.g.
    # Poe requires
    # `parameters` in tool definitions; Claude Code sometimes
    # omits it).
    # Check for provider-specific proxy or base proxy (poe2 ->
    # poe_proxy).
    # Proxies live in the INSTALLED share dir (install.sh copies
    # scripts/proxy/*.py
    # to $SHARED_DIR/proxy). This wrapper is self-contained in the
    # alias file and
    # has NO $LIB_DIR (that is a repo-only var), so resolve
    # against SHARED_DIR with
    # the same default lib.sh uses. Using $LIB_DIR here silently
    # disabled EVERY
    # proxy (e.g. Poe 400 "Invalid 'tools': Field required" when
    # Claude Code emits
    # a tool with no `parameters` – the poe_proxy injects it).
    local _cma_proxy_dir="${SHARED_DIR:-$HOME/.claude-shared}/
    proxy"
    local _base_id="${CMA_PROVIDER_ID%%[0-9]*}"
    local _proxy_script=""
    if [[ -x "$_cma_proxy_dir/${CMA_PROVIDER_ID}_proxy.py" ]];
    then
        _proxy_script=$_cma_proxy_dir/${CMA_PROVIDER_ID}_proxy.py"
    elif [[ -x "$_cma_proxy_dir/${_base_id}_proxy.py" ]]; then
        _proxy_script=$_cma_proxy_dir/${_base_id}_proxy.py"
    fi

```

```

if [[ -n "$_proxy_script" ]]; then
    local _proxy_port=3457
    python3 "$_proxy_script" --port "$_proxy_port" &
    _proxy_pid=$!
    local _waited=0
    while ! lsof -i :$_proxy_port >/dev/null 2>&1 && (( _waited
        < 25 )); do
        sleep 0.2
        _waited=$(( _waited + 1 ))
    done
    base="http://127.0.0.1:${_proxy_port}/v1/chat/completions"
    # cma_log is a lib.sh helper; the self-contained alias file
    # has no such
    # function, so guard the call to avoid a 'cma_log: command
    # not found' on
    # every proxied launch.
    command -v cma_log >/dev/null 2>&1 && cma_log "started proxy
    for $CMA_PROVIDER_ID on port $_proxy_port
    (pid=$_proxy_pid)" || true
fi
if command -v jq >/dev/null 2>&1; then
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")";
    chmod 600 "$tmp" 2>/dev/null || true
    # Pass the secret through the environment ($ENV.tok), never
    # as a jq argv
    # argument – argv is visible in ps/proc to other local
    # users.
    # `>|` (force-clobber), NOT `>`: cma_run_provider runs in
    # the user's
    # interactive shell, which may have `set -o noclobber`.
    # Plain `>` onto the
    # just-created mktemp file fails there ("cannot overwrite
    # existing file"),
    # silently dropping the router-config update so EVERY router
    # provider breaks.
    if CMA_TOK="$token" jq --arg n "$CMA_PROVIDER_ID" --arg u
    "$base" \
        --arg s "$CMA_PROVIDER_MODEL" --arg f "$
    {CMA_PROVIDER_FAST_MODEL:-$CMA_PROVIDER_MODEL}" '
    .Providers = ([ .Providers[]? | select(.name != $n) ]
    + [{name:$n, api_base_url:$u, api_key:$ENV.CMA_TOK,
    models:[$s,$f],
    transformer:{use:
    ["cleancache","streamoptions"]}]))
    | .Router.default = ($n + "," + $s)
    | .Router.background = ($n + "," + $f)
    ' "$cfg" >| "$tmp" 2>/dev/null; then
    mv "$tmp" "$cfg"; chmod 600 "$cfg" 2>/dev/null || true
    ccr restart >/dev/null 2>&1 || true
    else
    rm -f "$tmp"
    fi
fi
ccr code "$@"; rc=$?

```



```

# Stop proxy if we started one
if [[ -n "$_proxy_pid" ]]; then
    kill "$_proxy_pid" 2>/dev/null || true
    command -v cma_log >/dev/null 2>&1 && cma_log "stopped proxy
    for $CMA_PROVIDER_ID (pid=$_proxy_pid)" || true
fi
else
    export ANTHROPIC_BASE_URL="$CMA_PROVIDER_BASE_URL"
    export ANTHROPIC_AUTH_TOKEN="$token"
    export ANTHROPIC_MODEL="$CMA_PROVIDER_MODEL"
    [[ -n "${CMA_PROVIDER_FAST_MODEL:-}" ]] && export
        ANTHROPIC_SMALL_FAST_MODEL="$CMA_PROVIDER_FAST_MODEL"
    # Output-token guard: cap Claude Code's output at the provider
    model's real
    # max (limit.output). This is the OUTPUT half of the token-
    limit guard; the
    # INPUT half (CLAUDE_CODE_AUTO_COMPACT_WINDOW) is set above
    for both
    # transports. CMA_PROVIDER_MAX_OUTPUT comes from the catalog
    via
    # cma_provider_write_env.
    [[ -n "${CMA_PROVIDER_MAX_OUTPUT:-}" ]] && export
        CLAUDE_CODE_MAX_OUTPUT_TOKENS="$CMA_PROVIDER_MAX_OUTPUT"
    # Auto session-per-project (bare launch only – explicit args
    win verbatim).
    if [[ $# -eq 0 && -x "$HOME/.local/bin/claude-session" ]];
    then
        local _cma_psf
        _cma_psf="$("$HOME/.local/bin/claude-session" flags
        "$CLAUDE_CONFIG_DIR" 2>/dev/null || true)"
        "$HOME/.local/bin/claude-session" hint "$CMA_PROVIDER_ID"
        2>/dev/null || true
        eval "set -- $_cma_psf"
        # Auto-apply this provider alias's color to the session
        (idempotent).
        "$HOME/.local/bin/claude-session" apply-color
        "$CLAUDE_CONFIG_DIR" "$CMA_PROVIDER_ID" 2>/dev/null ||
        true
        _cma_pcolor=1
    fi
    "$CLAUDE_BIN" "$@"; rc=$?
fi
# Push post-session state back to all accounts/providers for
cross-alias visibility.
if [[ -x "$HOME/.local/bin/claude-sync-state" ]]; then
    "$HOME/.local/bin/claude-sync-state" push "$CLAUDE_CONFIG_DIR"
    2>/dev/null || true
fi
# Colour a freshly-created session too (its jsonl exists now),
so the colour is
# in place on the next resume even on the very first launch.
[[ "${_cma_pcolor:-}" == 1 && -x "$HOME/.local/bin/claude-
session" ]] && \

```

```

"$HOME/.local/bin/claude-session" apply-color
"$CLAUDE_CONFIG_DIR" "$CMA_PROVIDER_ID" 2>/dev/null ||
true
return $rc
}
EOF
fi
local rc src_line="source \"$ALIAS_FILE\""
# ${arr[@]+"${arr[@]}" } (not bare "${arr[@]}") so an EMPTY
CMA_RC_FILES does not
# trip "unbound variable" under `set -u` on bash 3.2 (macOS
ships 3.2; it errors
# on empty-array expansion where bash 4.4+ treats it as empty).
LOAD-BEARING.
for rc in ${CMA_RC_FILES[@]+"${CMA_RC_FILES[@]"}"; do
[[ -f "$rc" ]] || continue
cma_prune_stale_alias_sources "$rc" # self-heal: drop
dangling aliases.sh source lines
# Add the canonical source line only if no existing line
already sources THIS
# alias file (matched across .|source and $HOME/~/.absolute
forms) – prevents
# duplicate source lines accumulating across re-installs with
differing forms.
if ! cma_rc_sources_alias_file "$rc" "$ALIAS_FILE"; then
printf '\n# Claude multi-account aliases\n%s\n' "$src_line"
>> "$rc"
cma_log "added source line to $rc"
fi
done
}

# Add (or refresh) a single alias entry in $ALIAS_FILE.
Idempotent.
# The generated alias wraps `$CLAUDE_BIN` with sync-pull (before
launch) and
# sync-push (after exit) so the project/session index
inside .claude.json
# stays merged across every logged-in account without manual unify
runs.
cma_write_alias() {
local alias_name="$1" config_dir="$2"
cma_validate_alias "$alias_name"
# config_dir is interpolated into the alias body and re-parsed
by the shell
# when the alias is invoked. Reject shell metacharacters
(injection) and
# whitespace (an unquoted space would word-split the alias into
a bogus
# command – fail loud rather than write a silently-broken
alias). Matched
# literally per-char to avoid glob-bracket pitfalls.
local _cma_c
for _cma_c in "' '$' '`' '\\' ';' '&' '|' '<' '>' '(' ')'; do
case "$config_dir" in *"$_cma_c"*)

```

```

        cma_warn "refusing to write alias '$alias_name': unsafe
        config dir"
        return 1 ;;
    esac
done
case "$config_dir" in *[:space:]*)
    cma_warn "refusing to write alias '$alias_name': config dir
    must not contain whitespace"
    return 1 ;;
esac
cma_ensure_alias_file
# Strip any prior line for this alias, then append the new one.
local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
grep -v -E "^alias[:space:]+$alias_name=" "$ALIAS_FILE" >
"$tmp" || true
# Note: bash aliases can't take args, so we use a quoted
    CLAUDE_CONFIG_DIR= prefix
    # plus a wrapped invocation. The wrapper is a shell function
    reference (cma_run)
    # defined alongside in the alias file (added once by
    cma_ensure_alias_file).
printf 'alias %s="CLAUDE_CONFIG_DIR=%s cma_run"\n' \
"$alias_name" "$config_dir" >> "$tmp"
mv "$tmp" "$ALIAS_FILE"
}

# Remove an alias line. Idempotent.
cma_remove_alias() {
    local alias_name="$1"
    [[ -f "$ALIAS_FILE" ]] || return 0
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    grep -v -E "^alias[:space:]+$alias_name=" "$ALIAS_FILE" >
"$tmp" || true
    mv "$tmp" "$ALIAS_FILE"
}

# =====
# Provider-alias helpers (used by claude-providers.sh)
# =====

# The shared items every account/provider dir symlinks into
    $SHARED_DIR.
    # Kept here (single source) so claude-add-account and claude-
    providers agree.
CLAUDE_SHARED_ITEMS=(
    projects todos tasks plans file-history paste-cache shell-
    snapshots
    session-env telemetry sessions backups cache plugins
    stats-cache.json history.jsonl CLAUDE.md
)
# NOTE ($11.4 own-settings): settings.json is DELIBERATELY NOT in
    the shared set.
    # Each config dir gets its OWN settings.json so per-alias
    permissions/model/hooks

```

```

# never leak across aliases/providers, while the plugin CACHE
#   (`plugins`),
# history (`history.jsonl`), memory (`CLAUDE.md`) and sessions
#   stay shared. Each
# dir's own settings.json is seeded from + kept enabledPlugins-
#   synced with the
# shared template $SHARED_DIR/settings.json by
#   cma_own_settings_seed (so
# superpowers et al. stay enabled everywhere). See
#   cma_link_shared_items +
# cma_enable_plugins below.

cma_providers_dir() { echo "$HOME/.local/share/claude-multi-
    account/providers"; }

# --- verification status cache
# -----
# Single source of truth for "is this provider alias usable".
#   Holds ONLY
# non-secret metadata: provider id -> {status, model, checked_at,
#   failing_layer}.
# status is one of: verified | unverified | failed | pending.
#   Consumed by the
# list family (claude-providers list/list-all/list-faulty) and the
#   launch-time
# activation gate in cma_run_provider. NO key material ever lands
#   here.
cma_status_cache() { echo "$(cma_providers_dir)/status.json"; }

# cma_status_write <id> <status> [<model> [<failing_layer>]]
# Upserts one record. failing_layer is "" for verified/pending.
#   Atomic write.
cma_status_write() {
    local id="$1" status="$2" model="${3:-}" layer="${4:-}"
    cma_require jq
    local f; f="$(cma_status_cache)"; mkdir -p "$(dirname "$f")"
    [[ -s "$f" ]] || printf '{}\n' > "$f"
    # checked_at: portable UTC ISO-8601 (GNU + BSD date both accept
    #   -u +fmt).
    local now; now="$(date -u +%Y-%m-%dT%H:%M:%SZ)"
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if jq --arg id "$id" --arg s "$status" --arg m "$model" \
        --arg l "$layer" --arg t "$now" \
        '.[$id] = {status:$s, model:$m, checked_at:$t,
            failing_layer:$l}' \
        "$f" > "$tmp" 2>/dev/null; then
        mv "$tmp" "$f"
    else
        rm -f "$tmp"; cma_warn "could not update status cache $f"
    fi
}

# cma_status_read <id> -> status word (pending if absent/
#   unreadable).

```

```

cma_status_read() {
    local id="$1" f; f="$(cma_status_cache)"
    [[ -s "$f" ]] || { echo pending; return 0; }
    local s; s="$(jq -r --arg id "$id" '.[${id}].status // "pending"'
        "$f" 2>/dev/null)"
    [[ -n "$s" && "$s" != "null" ]] && echo "$s" || echo pending
}

# cma_status_all ->
#       id<TAB>status<TAB>model<TAB>checked_at<TAB>failing_layer
#       per record.
cma_status_all() {
    local f; f="$(cma_status_cache)"
    [[ -s "$f" ]] || return 0
    jq -r 'to_entries[] | [.key, .value.status, (.value.model //
        ""),
        (.value.checked_at // ""), (.value.failing_layer // "")]
        | @tsv' \
        "$f" 2>/dev/null || true
}

# Symlink every shared item into a config dir (account or
#       provider), creating
#       empty placeholders in $SHARED_DIR for any item that doesn't
#       exist yet.
# Idempotent: skips items already present in the target.
cma_link_shared_items() {
    local cdir="$1" item src tgt
    mkdir -p "$SHARED_DIR" "$cdir"
    for item in "${CMA_SHARED_ITEMS[@]}; do
        src="$SHARED_DIR/$item"; tgt="$cdir/$item"
        if [[ ! -e "$src" ]]; then
            case "$item" in
                *.json|*.jsonl|*.md) : > "$src" ;;
                *) mkdir -p "$src" ;;
            esac
        fi
        [[ -e "$tgt" || -L "$tgt" ]] || ln -s "$src" "$tgt"
    done

    # $11.4 own-settings: settings.json is NOT symlinked –
    #       give this dir its OWN copy.
    cma_own_settings_seed "$cdir"
}

# List all toolkit-managed config dirs (native accounts +
#       providers). Used to
#       fan out per-dir OWN settings.json enabledPlugins-sync.
cma_all_config_dirs() {
    local d
    for d in "$HOME"/.claude-claude* "$HOME"/.claude-prov-*; do
        [[ -d "$d" ]] && printf '%s\n' "$d"
    done
}

```

```

}

# Give a config dir its OWN settings.json ($11.4 own-settings). If
# it is a symlink
# (legacy shared layout) or absent, seed a REAL copy from the
# shared template so
# it inherits enabledPlugins + theme. If already a real file,
# additively merge the
# template's enabledPlugins in (own entries win – never clobber
# per-alias
# overrides). The shared template $SHARED_DIR/settings.json
# remains the single
# source of the always-on plugin set.
cma_own_settings_seed() {
    # NOTE: separate `local`s – a single `local a="$1" b="$a/x"`
    # expands $a before
    # it is assigned, which aborts under `set -u` ("a: unbound
    # variable").
    local cdir="${1:-}"
    [ -n "$cdir" ] || return 0
    local own="$cdir/settings.json"
    local tmp1="${SHARED_DIR:-$HOME/.claude-shared}/settings.json"
    local tmp=""
    command -v jq >/dev/null 2>&1 || return 0
    [[ -s "$tmp1" ]] || printf '{}\n' > "$tmp1"
    if [[ -L "$own" || ! -e "$own" ]]; then
        rm -f "$own" 2>/dev/null
        cp "$tmp1" "$own" 2>/dev/null || printf '{}\n' > "$own"
        return 0
    fi
    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    if jq -s '.[0] as $own | .[1] as $t
        | $own | .enabledPlugins = (($t.enabledPlugins // {})
        + ($own.enabledPlugins // {}))' \
        "$own" "$tmp1" > "$tmp" 2>/dev/null && jq -e . "$tmp" >/
        dev/null 2>&1; then
        mv "$tmp" "$own"
    else rm -f "$tmp"; fi
}

# Force-enable the always-on plugins in the shared settings.json
# enabledPlugins
# map (additive union – never removes a user's existing entries).
# Each arg is a
# plugin key as it appears in enabledPlugins (e.g.
# "superpowers@anthropics").
cma_enable_plugins() {
    cma_require jq
    local settings="$SHARED_DIR/settings.json" tmp
    mkdir -p "$SHARED_DIR"
    [[ -s "$settings" ]] || printf '{}\n' > "$settings"
    tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    local args=() p i=0

```

```

        # Use a dedicated counter for the jq --arg names: each
        # iteration appends THREE
        # elements (--arg, "pN", value), so deriving the index from $
        # {#args[@]} drifts
        # (it produced p0,p1,p3,p4 for 4 plugins → $p2 undefined → jq
        # failed silently

        # → no plugins enabled). The counter matches the $pN refs
        # in the prog below.
    for p in "$@"; do args+=(--arg "p$i" "$p"); i=$((i+1)); done
    # Build a jq program that sets each provided key true if absent.
    local prog='.enabledPlugins //= {}'
    local i=0
    for p in "$@"; do
        prog+=" | .enabledPlugins[\$p$i] //= true"; i=$((i+1))
    done
    # ${args[@]}+...} guards an empty array under set -u on bash 3.2
    # (reachable
    # via CMA_ALWAYS_ON_PLUGINS="" from non-re-exec'd claude-
    # providers.sh).
    if jq "${args[@]}+${args[@]}" "$prog" "$settings" > "$tmp" 2>/
    dev/null; then
        mv "$tmp" "$settings"
    else
        rm -f "$tmp"; cma_warn "could not update enabledPlugins in
        $settings"
    fi
    # Fan the always-on plugin set out into every managed config
    # dir's OWN
    # settings.json (settings.json is no longer a shared symlink –
    # $11.4
    # own-settings) so enabling a plugin still reaches every alias/
    # provider.
    local _cd
    while IFS= read -r _cd; do [[ -n "$_cd" ]] &&
        cma_own_settings_seed "$_cd"; done <
        <(cma_all_config_dirs)
}

# Write the non-secret per-provider env file consumed by
# cma_run_provider.
# Args: id keyvar transport base_url model fast_model config_dir
# [context_limit [max_output]]
# context_limit and max_output are optional trailing args (default
# empty).
# cma_run_provider consumes both catalog limits to avoid the 400
# "exceeded
# model token limit" error: CMA_PROVIDER_CONTEXT_LIMIT ->
# CLAUDE_CODE_AUTO_COMPACT_WINDOW
# (INPUT: compact before the prompt overshoots the provider's
# window) and
# CMA_PROVIDER_MAX_OUTPUT -> CLAUDE_CODE_MAX_OUTPUT_TOKENS (OUTPUT
# cap).
cma_provider_write_env() {

```

```

local id="$1" keyvar="$2" transport="$3" base="$4" model="$5"
    fast="$6" cdir="$7"
local context_limit="${8:-}" max_output="${9:-}"
    alias_name="${10:-}"
# Normalize the literal "null" (from a missing JSON field) to
empty so it

    # never leaks into the wrapper as a bogus value.
    transport+model were missed
# originally – a null strong_model/transport wrote
    CMA_PROVIDER_MODEL='null'
# (provider launches with a bogus model). Normalize every field
for symmetry.
[[ "$transport" == "null" ]] && transport=""
[[ "$base" == "null" ]] && base=""
[[ "$model" == "null" ]] && model=""
[[ "$fast" == "null" ]] && fast=""
[[ "$context_limit" == "null" ]] && context_limit=""
[[ "$max_output" == "null" ]] && max_output=""
[[ "$alias_name" == "null" ]] && alias_name=""
local pdir; pdir="$(cma_providers_dir)"; mkdir -p "$pdir"
# Values are single-quoted (with embedded-quote escaping) so
sourcing the file
# in the user's shell is safe regardless of characters in URLs/
model ids.
# POSIX single-quote escaping: replace each ' with '\'' via a
loop.
# Portable across all bash versions – avoids ${var/pattern/
replacement} with
# complex escape sequences that break on bash 3.2 and 5.3.
_cma_q() {
    local _r="" _rem="$1"
    while true; do
        case "$_rem" in *\'*)
            _r="${_r}${_rem%%\'*}'\''"; _rem="${_rem#*\'}" ;;
        *) _r="${_r}${_rem}"; break ;;
        esac
    done
    printf "%s" "$_r"
}
cat > "$pdir/$id.env" <<EOF
# generated by claude-providers – non-secret. Do not edit by hand.
# Secrets are NEVER stored here; the key is read from the keys
file at launch.
CMA_PROVIDER_ID=$( _cma_q "$id")
CMA_PROVIDER_KEYVAR=$( _cma_q "$keyvar")
CMA_PROVIDER_TRANSPORT=$( _cma_q "$transport")
CMA_PROVIDER_BASE_URL=$( _cma_q "$base")
CMA_PROVIDER_MODEL=$( _cma_q "$model")
CMA_PROVIDER_FAST_MODEL=$( _cma_q "$fast")
CMA_PROVIDER_CONFIG_DIR=$( _cma_q "$cdir")
# Context-window limits from the models.dev catalog for the
selected strong model.

```



```

# CMA_PROVIDER_CONTEXT_LIMIT: input context window (tokens); empty
    = unknown.
#   -> exported as CLAUDE_CODE_AUTO_COMPACT_WINDOW (input-side
    guard).
# CMA_PROVIDER_MAX_OUTPUT:    maximum output tokens; empty =
    unknown.
#   -> exported as CLAUDE_CODE_MAX_OUTPUT_TOKENS (output-side
    guard).
CMA_PROVIDER_CONTEXT_LIMIT=$(_cma_q "$context_limit")
CMA_PROVIDER_MAX_OUTPUT=$(_cma_q "$max_output")
# Alias name for this provider (used by 'list --refresh-aliases'
    to rebuild the
# alias shell line with NO network – the session hook's fast
    path). Empty is OK;
# refresh falls back to the provider id as the alias name.
CMA_PROVIDER_ALIAS=$(_cma_q "$alias_name")
EOF
    unset -f _cma_q
}

# Write (or refresh) a provider alias: alias
    <name>="cma_run_provider <id>".
cma_provider_write_alias() {
    local alias_name="$1" id="$2"
    cma_validate_alias "$alias_name"
    # The provider id is interpolated into the alias body and re-
        parsed when the
    # alias is invoked. Provider ids are always [A-Za-z0-9._-];
        reject anything
    # else so a hostile catalog/--id value can't inject shell
        commands.
    case "$id" in
        ''|*[^A-Za-z0-9._-]*)
            cma_warn "refusing to write alias '$alias_name': unsafe
                provider id"
            return 1 ;;
    esac
    # Only BOOTSTRAP the alias file when it is absent – do NOT re-
        run the full

        # cma_ensure_alias_file (header + self-heal migrations) on
        every alias write.

        # Those migrations are install/invoke-time concerns;
        running them per
    # alias line made '--refresh-aliases' non-idempotent – a
        migration could

        # reposition the cma_run_provider function relative to the
        alias lines
    # (body byte-identical, only its position moved), so a second
        refresh no
    # longer produced an identical file ($ idempotence; see
        test_providers.sh

```

```

# "--refresh-aliases is idempotent"). Existence is all this
# function needs.
[[ -f "$ALIAS_FILE" ]] || cma_ensure_alias_file
local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
grep -v -E "^alias[[:space:]]+${alias_name}=" "$ALIAS_FILE" >
"$tmp" || true
printf 'alias %s="cma_run_provider %s"\n' "$alias_name" "$id"
>> "$tmp"
mv "$tmp" "$ALIAS_FILE"
}

# Install (idempotently) the provider session-refresh hook into
# $ALIAS_FILE. On
# every interactive shell start the hook re-writes provider
# aliases from cache
# (NO network) and, when the status cache is older than
# CMA_PROVIDERS_SYNC_TTL
# (default 24h), kicks a detached full sync ($11.4.89 background -
# never blocks
# the shell). Bracketed by markers so a re-install replaces the
# block atomically
# (no duplication across re-installs).
cma_install_session_hook() {
    cma_ensure_alias_file
    local begin='# cma-providers-session-refresh BEGIN'
    local end='# cma-providers-session-refresh END'
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    # Drop any existing block (BEGIN..END inclusive), then append
    # the fresh one.
    awk -v b="$begin" -v e="$end" '
        $0==b{skip=1} !skip{print} $0==e{skip=0}' "$ALIAS_FILE" >
        "$tmp"
    {
        printf '%s\n' "$begin"
        cat <<'HOOK'
cma_providers_session_refresh() {
    command -v claude-providers >/dev/null 2>&1 || return 0
    # No-network: re-write alias functions from the cached env
    # files.
    claude-providers list --quiet --refresh-aliases >/dev/null 2>&1
    || true
    # TTL-triggered background full sync (detached; never blocks the
    # shell).
    local ttl="${CMA_PROVIDERS_SYNC_TTL:-86400}"
    local sf="$HOME/.local/share/claude-multi-account/providers/
    status.json"
    if [ -f "$sf" ]; then
        local now mtime age
        now="$(date +%s)"
        mtime="$(date -r "$sf" +%s 2>/dev/null || stat -c %Y "$sf" 2>/
        dev/null || echo "$now")"
        age=$(( now - mtime ))
        if [ "$age" -gt "$ttl" ]; then

```

```

        ( nohup claude-providers sync >/dev/null 2>&1 & disown ) 2>/
        dev/null || true
    fi
fi
}
cma_providers_session_refresh
HOOK
    printf '%s\n' "$end"
} >> "$tmp"
mv "$tmp" "$ALIAS_FILE"
}

# True only when the toolkit may prompt the user interactively.
#   Scripts read
# confirmations from /dev/tty (so prompts survive `curl | bash`),
#   so this
# probes /dev/tty rather than stdin. It returns false when:
#   * CMA_NONINTERACTIVE=1 is exported – a global "never prompt"
#     switch for
#     automation, CI, and the test suite (deterministic regardless
#     of TTY); or
#   * no terminal is available – CI, a test sandbox, an SSH
#     command with no PTY.
# When it returns false, callers MUST fall back to their non-
# interactive
# default instead of blocking or erroring on a failed read. This
# is what makes
# toolkit execution always non-interactive off a terminal.
cma_can_prompt() {
    [[ "${CMA_NONINTERACTIVE:-}" == 1 ]] && return 1
    ( exec </dev/tty ) 2>/dev/null
}

```

8.2 claude-unify.sh

```

#!/usr/bin/env bash
# claude-unify.sh – Unify N Claude Code account config dirs into a
#   single
# shared store so every account sees the same projects/
#   conversations,
# memory, history, todos, plans, plugins, and settings.
#
# Inputs:
#   * Positional args: one or more account config directories. If
#     none
#     are given, all `~/.claude-*` directories are auto-detected
#     (excluding
#     the shared store itself).
#   * Env vars:
#     SHARED_DIR    target shared store (default: ~/.claude-
#                   shared)
#     DEFAULT_DIR   user-scope plugin root (default: ~/.claude)
#

```

```

# Use --rollback to restore the .preunify.* backups and remove the
# shared
# store. Safe to re-run any time.

set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. Re-exec under
# Homebrew bash
# if available; otherwise tell the user how to install it.
if (( BASH_VERSION[0] < 4 )); then
    for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
        [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
    done
    echo "claude-unify requires bash 4+. Install via: brew install
        bash" >&2
    exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DEFAULT_DIR="${DEFAULT_DIR:-$HOME/.claude}"

cma_require rsync
cma_require jq
cma_require awk

# Items each account dir contributes to the shared store. Order
# matters
# only insofar as we want plugins last so the path-rewrite step
# has all
# manifest entries to rewrite at once.
SHARED_ITEMS=(
    projects
    todos
    tasks
    plans
    file-history
    paste-cache
    shell-snapshots
    session-env
    telemetry
    sessions

```

```

backups
cache
stats-cache.json
history.jsonl
plugins
)
# NOTE ($11.4 own-settings): settings.json is DELIBERATELY NOT in
    SHARED_ITEMS,
# so it is never symlinked into an account dir. Unify does NOT
    blanket-merge full
# account settings into the shared store (that re-leaks per-alias
    permissions/
# model/hooks to shared and thence to every dir). Instead it ONLY
    unions the
# enabledPlugins map into the shared TEMPLATE
    (union_enabled_plugins_into_template)
# and then re-seeds each dir's OWN settings.json from it
    (seed_own_settings) so
# newly-enabled plugins propagate while every other per-alias key
    stays local.
# This list MUST stay == CMA_SHARED_ITEMS (lib.sh) minus the
    CLAUDE.md special-
# case (enforced by the drift-guard test in test_unify.sh).

PRIVATE_ITEMS=(
    .credentials.json
    .claude.json
    mcp-needs-auth-cache.json
)

ts() { date +%Y%m%d%H%M%S; }

already_linked_to_shared() {
    [[ -L "$1" ]] || return 1
    # cma_realpath (not `readlink -f`, which is absent on BSD/macOS
    #   – there the
    # old check always failed, so unify re-linked every item on
    #   every re-run).
    [[ "$(cma_realpath "$1")" == "$(cma_realpath "$2")" ]]
}

backup_and_remove() {
    local p="$1"
    [[ -e "$p" || -L "$p" ]] || return 0
    mv "$p" "${p}.preunify.${ts}"
}

link_to_shared() {
    local item="$1" acct
    # Never create a dangling symlink: if the merge step did not
    #   produce the
    # shared item (e.g. settings.json skipped because every account
    #   file was

```

```

# invalid JSON, or a stats-cache.json that no account had),
# leave each
# account's real file untouched rather than backing it up and
# pointing it at a
# missing target – that would silently replace a valid live
# config with a
# broken link.
[[ -e "$SHARED_DIR/$item" ]] || return 0
for acct in "${ACCOUNTS[@]"; do
    [[ -d "$acct" ]] || continue
    local target="$acct/$item"
    already_linked_to_shared "$target" "$SHARED_DIR/$item" &&
        continue
    backup_and_remove "$target"
    mkdir -p "$(dirname "$target")"
    ln -s "$SHARED_DIR/$item" "$target"
done
}

merge_dir_into_shared() {
    local item="$1" acct rc
    mkdir -p "$SHARED_DIR/$item"
    # First pass: each account fills only gaps (--ignore-existing)
    # so the
    # union is preserved across N accounts.
    for acct in "${ACCOUNTS[@]"; do
        if [[ -d "$acct/$item" && ! -L "$acct/$item" ]]; then
            rc=0
            rsync -a --ignore-existing "$acct/$item/" "$SHARED_DIR/
                $item/" || rc=$?
            # rsync exit 23/24 are partial transfers ("some files
            # vanished" / "some
            # files couldn't be transferred") that we treat as warnings
            # – common on
            # macOS for `unlinkat: Directory not empty` when symlinks
            # straddle the
            # tree. Anything else is fatal.
            (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
        fi
    done

    # Second pass: overlay every account with rsync -u
    # (update-only) so the file
    # carrying the NEWEST mtime wins each conflict, regardless of
    # how many accounts
    # there are or what they're named. The previous code overlaid
    # only
    # ACCOUNTS[-1] – the alphabetically-last dir, not necessarily
    # the most recently
    # active one – which let a stale lexically-last account clobber
    # fresher content
    # (e.g. memory/*.md) from an earlier account.
    for acct in "${ACCOUNTS[@]"; do
        if [[ -d "$acct/$item" && ! -L "$acct/$item" ]]; then

```

```

    rc=0
    rsync -au "$acct/$item/" "$SHARED_DIR/$item/" || rc=$?
    (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
fi
done
}

merge_history_jsonl() {
    local acct srcs=() tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    for acct in "${ACCOUNTS[@]}; do
        local f="$acct/history.jsonl"
        if [[ -f "$f" && ! -L "$f" ]]; then srcs+=("$f"); fi
    done
    if [[ -f "$SHARED_DIR/history.jsonl" && ! -L "$SHARED_DIR/history.jsonl" ]]; then
        srcs+=("$SHARED_DIR/history.jsonl")
    fi
    # Feed the files straight to awk rather than cat'ing them
    # together first: awk
    # starts a fresh record at every file boundary, so a source
    # missing its
    # trailing newline cannot fuse its last line onto the next
    # file's first line
    # (plain `cat` would). Write via a temp file so we never
    # truncate shared while
    # it is also one of the inputs.
    if (( ${#srcs[@]} )); then
        awk 'NF && !seen[$0]++' "${srcs[@]}" > "$tmp"
        mv "$tmp" "$SHARED_DIR/history.jsonl"
    else
        : > "$SHARED_DIR/history.jsonl"
        rm -f "$tmp"
    fi
    return 0
}

# §11.4 own-settings: unify NEVER blanket-merges full account
# settings into the
# shared store – that would re-leak each alias's permissions/
# model/hooks to
# shared and, through the shared symlink, to every other alias.
# Instead it ONLY
# extracts the enabledPlugins map from every account's OWN
# settings.json and
# UNIONS it (any-true) into the shared TEMPLATE's enabledPlugins,
# leaving every
# other key of the template – and every key of each dir's OWN
# settings.json –
# untouched. seed_own_settings then propagates that union back
# into each dir.

union_enabled_plugins_into_template() {
    local acct files=() resolved f tpl="$SHARED_DIR/settings.json"
    for acct in "${ACCOUNTS[@]}; do

```

```

f="$acct/settings.json"
if [[ -L "$f" ]]; then
    resolved="$(cma_realpath "$f")" # readlink -f is
    unavailable on BSD/macOS
    if [[ -f "$resolved" ]]; then files+=("$resolved"); fi
elif [[ -f "$f" ]]; then
    files+=("$f")
fi
done
# Include the existing template so a plugin enabled on a prior
# run (any-true)
# persists even if no current account carries it.
[[ -f "$tpl" ]] && files+=("$tpl")
# De-duplicate. On re-runs two account paths can resolve to the
# same file, and

# passing the same path twice would double-count (harmless
# for a union, but we
# keep the set clean).
if (( ${#files[@]} > 1 )); then
    mapfile -t files <<(printf '%s\n' "${files[@]}" | awk '!
    seen[$0]++')
fi
# Drop any file that is not valid JSON so one hand-edited/
# corrupt account
# settings.json can't sink the whole union – jq -s slurps every
# file and fails
# wholesale on a single parse error. See test R3.
local validf=()
for f in "${files[@]}"; do
    if jq empty "$f" 2>/dev/null; then
        validf+=("$f")
    else
        cma_warn "settings.json in $(dirname
        "$f") is not valid JSON – excluded from enabledPlugins
        union"
    fi
done
files=("${validf[@]}")
(( ${#files[@]} )) || return 0
mkdir -p "$SHARED_DIR"
[[ -f "$tpl" ]] || printf '{}\n' > "$tpl"
local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"

# Two-step so the shared template's OWN non-plugin keys
# (if any) survive and no
# per-account non-plugin key ever enters shared:
# 1. build the any-true union of enabledPlugins across every
# valid file,
# with keys SORTED so the template is byte-stable across
# re-runs
# regardless of per-account key order (idempotency, test
# line ~93);

```



```

        # 2. set ONLY .enabledPlugins on the existing template
        to that union.
local union
if union="$(jq -s '
    reduce .[] as $x ({};
        reduce (($x.enabledPlugins // {}) | to_entries[]) as $e
        (.;
            .[$e.key] = (([$e.key] // false) or $e.value)))
    | to_entries | sort_by(.key) | from_entries
    ' "${files[@]}" 2>/dev/null)" \
&& printf '%s' "$union" | jq empty 2>/dev/null \
&& jq --argjson u "$union" '.enabledPlugins = $u' "$tpl" >
    "$tmp" 2>/dev/null \
&& jq -e . "$tmp" >/dev/null 2>&1; then
    mv "$tmp" "$tpl"
else
    rm -f "$tmp"
    cma_warn "settings.json: enabledPlugins union skipped (jq
        error)"
fi
return 0
}

# Give every account dir its OWN real settings.json ($11.4 own-
# settings): seed
# from the shared TEMPLATE when absent/legacy-symlink, else
# additively merge the
# template's enabledPlugins in (own keys always win). Per-alias
# non-plugin keys
# stay local – they never leak to shared or to a sibling dir.
# Delegates to the
# single canonical seeder in lib.sh so unify/add-account/
# providers/bootstrap all
# agree.
seed_own_settings() {
    local acct
    for acct in "${ACCOUNTS[@]"; do
        [[ -d "$acct" ]] || continue
        cma_own_settings_seed "$acct"
    done
}

merge_file_into_shared() {
    local item="$1" acct newest=""
    for acct in "${ACCOUNTS[@]"; do
        local f="$acct/$item"
        if [[ -L "$f" ]]; then continue; fi
        if [[ ! -f "$f" ]]; then continue; fi
        if [[ -z "$newest" || "$f" -nt "$newest" ]]; then
            newest="$f"; fi
        done
        if [[ -n "$newest" ]]; then cp -p "$newest" "$SHARED_DIR/
            $item"; fi

```

```

    return 0
}

absorb_default_plugins() {
    [[ -d "$DEFAULT_DIR/plugins" && ! -L "$DEFAULT_DIR/plugins" ]]
    || return 0
    mkdir -p "$SHARED_DIR/plugins"
    local rc=0
    rsync -a --ignore-existing "$DEFAULT_DIR/plugins/" "$SHARED_DIR/
        plugins/" || rc=$?
    (( rc == 0 || rc == 23 || rc == 24 )) || return $rc
    return 0
}

rewrite_plugin_paths() {
    local f="$SHARED_DIR/plugins/installed_plugins.json"
    if [[ -f "$f" ]]; then
        local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
        jq --arg shared "$SHARED_DIR" '
            .plugins |= with_entries(
                .value |= map(
                    if ((.installPath // "") | test(".*plugins/cache/"))
                    then .installPath = ($shared + "/plugins/cache/" +
                        (.installPath | sub(".*plugins/cache/"; "")))
                    else . end
                )
            )' "$f" > "$tmp" && mv "$tmp" "$f"
    fi
    local km="$SHARED_DIR/plugins/known_marketplaces.json"
    if [[ -f "$km" ]]; then
        local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
        jq --arg shared "$SHARED_DIR" '
            with_entries(
                if ((.value.installLocation // "") | test(".*plugins/
                    marketplaces/"))
                then .value.installLocation = ($shared + "/plugins/
                    marketplaces/" + (.value.installLocation | sub(".*
                    plugins/marketplaces/"; "")))
                else . end
            )' "$km" > "$tmp" && mv "$tmp" "$km"
    fi
}

# Repoint ~/.claude/plugins/cache and ~/.claude/plugins/
# marketplaces at
# the shared store so future user-scope installs land in one
# place.
link_default_plugin_subdirs() {
    for sub in cache marketplaces; do
        local src="$SHARED_DIR/plugins/$sub" tgt="$DEFAULT_DIR/
            plugins/$sub"
        [[ -d "$src" ]] || continue
        already_linked_to_shared "$tgt" "$src" && continue
    done
}

```

```

    mkdir -p "$DEFAULT_DIR/plugins"
    backup_and_remove "$tgt"
    ln -s "$src" "$tgt"
done
}

# CLAUDE.md (user-scope memory) lives at ~/.claude/CLAUDE.md by
# default.
# We promote it to the shared store and symlink all three
# locations (each
# account dir + ~/.claude) so any account writing user memory
# updates the
# same file.
sync_claude_md() {
    local shared_md="$SHARED_DIR/CLAUDE.md" src
    if [[ ! -f "$shared_md" ]]; then
        if [[ -f "$DEFAULT_DIR/CLAUDE.md" && ! -L "$DEFAULT_DIR/
CLAUDE.md" ]]; then
            cp -p "$DEFAULT_DIR/CLAUDE.md" "$shared_md"
        else
            for acct in "${ACCOUNTS[@]"; do
                if [[ -f "$acct/CLAUDE.md" && ! -L "$acct/CLAUDE.md" ]];
                then
                    cp -p "$acct/CLAUDE.md" "$shared_md"; break
                fi
            done
        fi
        [[ -f "$shared_md" ]] || return 0
    fi
    local targets=("$DEFAULT_DIR/CLAUDE.md")
    for acct in "${ACCOUNTS[@]"; do targets+=("$acct/CLAUDE.md");
    done
    for tgt in "${targets[@]"; do
        [[ -d "$(dirname "$tgt")" ]] || continue
        already_linked_to_shared "$tgt" "$shared_md" && continue
        backup_and_remove "$tgt"
        ln -s "$shared_md" "$tgt"
    done
}

rollback() {
    cma_log "rollback: restoring .preunify.* backups"
    local root
    local roots=("$DEFAULT_DIR" "$DEFAULT_DIR/plugins"
"$SHARED_DIR")
    for root in "${ACCOUNTS[@]"; do roots+=("$root"); done
    for root in "${roots[@]"; do
        [[ -d "$root" ]] || continue

        # Restore backups oldest-first. The suffix is a
        # YYYYMMDDHHMMSS timestamp, so
        # `sort -z` orders lexically = chronologically; the earliest
        # backup is the

```

```

# true pre-unify original and wins when a path has more than
# one .preunify.*.
while IFS= read -r -d '' bk; do
    local orig="${bk%.preunify.*}"
    [[ -L "$orig" ]] && rm -f "$orig"
    [[ -e "$orig" ]] || { mv "$bk" "$orig"; cma_log "restored
    $orig"; }
done <<(find "$root" -maxdepth 1 -name '*.preunify.*'
-print0 2>/dev/null | sort -z)
# Don't scan the shared store itself (its contents move aside
# intact below);
# only external roots can hold links that would dangle.
[[ "$root" == "$SHARED_DIR" ]] && continue
# Remove leftover symlinks pointing into the shared store.
# unify created
# these for items an account never had (so there's no backup
# to restore);
# without this they dangle once SHARED_DIR is moved aside.
while IFS= read -r -d '' lnk; do
    local tgt; tgt="$(readlink "$lnk")"
    case "$tgt" in
        "$SHARED_DIR"|"${SHARED_DIR}/") rm -f "$lnk"; cma_log
        "removed shared-store symlink $lnk" ;;
    esac
done <<(find "$root" -maxdepth 1 -type l -print0 2>/dev/null)
done
if [[ -d "$SHARED_DIR" ]]; then
    mv "$SHARED_DIR" "${SHARED_DIR}.removed.${ts}"
    cma_log "moved $SHARED_DIR aside"
fi
}

usage() {
    cat <<EOF
Usage: $(basename "$0") [--rollback] [account-dir ...]

Without args, auto-detects all ~/${ACCOUNT_PREFIX}* directories.
With --rollback, restores .preunify.* backups and removes the
    shared store.

Env: SHARED_DIR=$SHARED_DIR DEFAULT_DIR=$DEFAULT_DIR
EOF
}

# === main ===

ACCOUNTS=()
DO_ROLLBACK=0
while (( $# )); do
    case "$1" in
        -h|--help) usage; exit 0 ;;
        --rollback) DO_ROLLBACK=1; shift ;;
        --) shift; while (( $# )); do ACCOUNTS+=("$1"); shift; done ;;
    esac
done

```

```

    *) cma_die "unknown flag: $1" ;;
    *) ACCOUNTS+=("$1"); shift ;;
esac
done

if (( ${#ACCOUNTS[@]} == 0 )); then
    mapfile -t ACCOUNTS < <(cma_detect_accounts)
fi

(( ${#ACCOUNTS[@]} >= 1 )) || cma_die "no account dirs given and
    none auto-detected at ~/${ACCOUNT_PREFIX}*"

cma_log "accounts: ${ACCOUNTS[*]}"
cma_log "shared:   $SHARED_DIR"

if (( DO_ROLLBACK )); then rollback; exit 0; fi

mkdir -p "$SHARED_DIR"
absorb_default_plugins

for item in "${SHARED_ITEMS[@]"; do
    case "$item" in
        history.jsonl)    merge_history_jsonl ;;
        stats-cache.json) merge_file_into_shared "$item" ;;
        *)                merge_dir_into_shared "$item" ;;
    esac
    [[ "$item" == "plugins" ]] && rewrite_plugin_paths
    link_to_shared "$item"
    cma_log "ok: $item"
done

# $11.4 own-settings: settings.json is NOT a shared symlink. Union
# each account's
# enabledPlugins into the shared template, then re-seed every
# dir's OWN copy so
# the union propagates while per-alias non-plugin keys stay local.
union_enabled_plugins_into_template
seed_own_settings
cma_log
    "ok: settings.json (per-dir OWN copy; enabledPlugins union
    propagated; per-alias keys kept local)"

link_default_plugin_subdirs
sync_claude_md

# Merge the projects/session index inside .claude.json across
# every account.
# This is the single most important step for cross-account session
# resume:
# without it, account A's session UUIDs are invisible to account B
# even though
# the JSONL transcripts are already shared on disk via projects/.
cma_merge_claude_json "${ACCOUNTS[@]}"

```

```

cma_log "ok: .claude.json (projects/session index merged; auth
keys preserved per-account)"

# Ensure every detected account has an invocable alias. install.sh
and direct
# unify runs may discover pre-existing ~/.claude-* dirs that were
never
# registered (e.g. the user ran install.sh after creating the
dirs). Without
# this step `claude1` etc. are undefined even though the dirs
exist.
cma_ensure_alias_file
declare -A ALIAS_OF_DIR=()
if [[ -f "$ALIAS_FILE" ]]; then
    while IFS= read -r line; do
        [[ "$line" =~ ^alias[[:space:]]+([a-zA-Z0-9_-]
+)=.*CLAUDE_CONFIG_DIR=([^[:space:]]+) ]] || continue
        ALIAS_OF_DIR["${BASH_REMATCH[2]}"]="${BASH_REMATCH[1]}"
    done < "$ALIAS_FILE"
fi
# First pass: dirs whose suffix (after ~/.claude-) is already
"claude<N>"
# (e.g. ~/.claude-claude4) keep that number, so user expectations
about named
# dirs are preserved.
for acct in "${ACCOUNTS[@]"; do
    if [[ -z "${ALIAS_OF_DIR[$acct]:-}" ]]; then
        base="$(basename "$acct")"
        suffix="${base}${ACCOUNT_PREFIX}"
        if [[ "$suffix" =~ ^claude([0-9]+)$ ]]; then
            wanted="claude${BASH_REMATCH[1]}"
            if ! cma_existing_aliases | grep -qx "$wanted"; then
                cma_write_alias "$wanted" "$acct"
                cma_log "registered alias: $wanted -> $acct"
                continue
            fi
        fi
    fi
done
# Lowest free claude<N> (fills gaps left by explicit claudeM
basenames).
_cma_lowest_free_claude() {
    local n=1
    while cma_existing_aliases | grep -qx "claude$n"; do
        n=$((n + 1))
        (( n < 1000 )) || { cma_warn "could not find a free claude<N>
alias"; return 1; }
    done
    printf 'claude%s\n' "$n"
}

# Second pass: anything still unaliased gets the lowest free
claude<N>.
for acct in "${ACCOUNTS[@]"; do

```

```

if [[ -z "${ALIAS_OF_DIR[$acct]:-}" ]]; then
    if grep -qE "alias[[:space:]]+[^=]
        +=.*CLAUDE_CONFIG_DIR=$acct([[:space:]]|$)"
        "$ALIAS_FILE"; then
        # A newly-written pass-1 alias now points at this dir;
        update our map.
        ALIAS_OF_DIR[$acct]="$(grep -E "alias[[:space:]]+[^=]
            +=.*CLAUDE_CONFIG_DIR=$acct([[:space:]]|$)" "$ALIAS_FILE"
            | head -1 | sed -E 's/^alias[[:space:]]+([^\=]+)=.*\/\1\/')"
        continue
    fi
    next_alias="$_cma_lowest_free_clauden)"
    cma_write_alias "$next_alias" "$acct"
    cma_log "registered alias: $next_alias -> $acct"
fi
done

cma_log "done. shared: $SHARED_DIR"
cma_log "private per-account: ${PRIVATE_ITEMS[*]}"

```

8.3 claude-add-account.sh

```

#!/usr/bin/env bash
# claude-add-account.sh – Add a new Claude Code account to the
# multi-account
# setup. Creates the per-account config directory, links every
# shared item
# to the shared store, and registers a shell alias so the new
# account is
# invocable as e.g. `claude3`.
#
# Modes:
# * Interactive (default): prompts for alias name and config dir
#   name,
#   showing defaults that follow the existing pattern.
# * Non-interactive: pass --alias NAME and optionally --dir
#   PATH, e.g.
#   `claude-add-account.sh --alias work --dir ~/.claude-work`.
#
# After this script runs you still need to authenticate the new
# account
# with Anthropic – the script prints the exact command for that.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done

```

```

LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
CONFIG_DIR=""
NONINTERACTIVE=0

usage() {
    cat <<EOF
Usage: $(basename "$0") [--alias NAME] [--dir PATH] [--yes]

    --alias NAME    Shell alias to create (e.g. claude3 or work).
                    Default:
                        next free claudeN.
    --dir    PATH    Config directory for the new account. Default:
                    ~/${ACCOUNT_PREFIX}<alias>.
    --yes          Skip prompts and use defaults / passed values.
EOF
}

while (( $# )); do
    case "$1" in
        -h|--help) usage; exit 0 ;;
        --alias) ALIAS_NAME="$2"; shift 2 ;;
        --dir) CONFIG_DIR="$2"; shift 2 ;;
        --yes|-y) NONINTERACTIVE=1; shift ;;
        *) cma_die "unknown arg: $1" ;;
    esac
done

# Prompt with a default value, return whatever the user types (or default).
prompt() {
    local label="$1" default="$2" answer
    # Use the default when --yes was passed OR no terminal is available to
    # prompt from (CI, test sandbox, SSH without a PTY) – never block.
    if (( NONINTERACTIVE )) || ! cma_can_prompt; then echo "$default"; return; fi
    read -r -p "$label [$default]: " answer < /dev/tty
    echo "${answer:-$default}"
}

[[ -n "$ALIAS_NAME" ]] || ALIAS_NAME="$(cma_suggest_alias)"
ALIAS_NAME="$(prompt "Alias name" "$ALIAS_NAME")"
cma_validate_alias "$ALIAS_NAME"

# Reject if alias already exists in the alias file.
if cma_existing_aliases | grep -qx "$ALIAS_NAME"; then
    cma_die "alias '$ALIAS_NAME' already exists in $ALIAS_FILE"

```



```

fi

[[ -n "$CONFIG_DIR" ]] || CONFIG_DIR="$HOME/${ACCOUNT_PREFIX}${
    {ALIAS_NAME}"
CONFIG_DIR="$(prompt "Config directory" "$CONFIG_DIR")"
case "$CONFIG_DIR" in /*) ;; *) CONFIG_DIR="$HOME/$CONFIG_DIR" ;;
esac

[[ -d "$CONFIG_DIR" ]] && cma_die "config dir already exists:
    $CONFIG_DIR (refusing to overwrite)"

mkdir -p "$CONFIG_DIR"
cma_log "created $CONFIG_DIR"

# Symlink every shared item into the new account dir, using the
    single
# canonical list (CMA_SHARED_ITEMS in lib.sh) so accounts and
    provider
# aliases stay in lockstep without two copies of the list drifting
    apart.
cma_link_shared_items "$CONFIG_DIR"
cma_log "linked ${#CMA_SHARED_ITEMS[@]} shared items into
    $CONFIG_DIR"

# Add the alias. lib.sh handles dedupe, rc-file sourcing, etc.
cma_write_alias "$ALIAS_NAME" "$CONFIG_DIR"
cma_log "registered alias: $ALIAS_NAME -> $CONFIG_DIR"

cat <<EOF

[done] new account: $ALIAS_NAME

Next steps:
1. Reload your shell (or run: source $ALIAS_FILE).
2. Authenticate the new account:
    $ALIAS_NAME /login
3. After login, run any project – history, memory, plugins, and
    settings will already match your other accounts via
    $SHARED_DIR.

To remove this account later:
    $LIB_DIR/claude-remove-account.sh --alias $ALIAS_NAME
EOF

```

8.4 claude-remove-account.sh

```

#!/usr/bin/env bash
# claude-remove-account.sh – Remove a Claude Code account from the
# multi-account setup. Drops the alias from the managed alias file
    and
# (optionally) deletes or archives the per-account config
    directory.
#

```

```

# The shared store is left untouched – only this one account's
  symlinks
# and credentials are removed. Other accounts continue using
  shared.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
  ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
    (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

ALIAS_NAME=""
DELETE_DIR=0
NONINTERACTIVE=0

usage() {
  cat <<EOF
Usage: $(basename "$0") --alias NAME [--delete | --archive] [--
  yes]

  --alias NAME    Required. Alias to remove (e.g. claude3).
  --delete        Permanently delete the per-account config
                  directory.
  --archive       Move the per-account dir to
                  <dir>.removed.<timestamp>
                  instead of deleting (default).
  --yes           Skip confirmation prompts.
EOF
}

while (( $# )); do
  case "$1" in
    -h|--help) usage; exit 0 ;;
    --alias) ALIAS_NAME="$2"; shift 2 ;;
    --delete) DELETE_DIR=1; shift ;;
    --archive) DELETE_DIR=0; shift ;;
    --yes|-y) NONINTERACTIVE=1; shift ;;
    *) cma_die "unknown arg: $1" ;;
  esac
done

[[ -n "$ALIAS_NAME" ]] || { usage; exit 2; }
cma_validate_alias "$ALIAS_NAME"

```

```

# Resolve the config dir by inspecting the existing alias line.
CONFIG_DIR=""
if [[ -f "$ALIAS_FILE" ]]; then
    # POSIX/2-arg awk match() only: the 3-arg capture form is GNU-
    # awk-specific

    # and silently yields an empty CONFIG_DIR on the BSD awk
    # shipped with macOS.
    CONFIG_DIR="$(awk -v a="$ALIAS_NAME" '
        $0 ~ "^alias[[:space:]]+a=\"" {
            if (match($0, /CLAUDE_CONFIG_DIR=[^ ]+/)) {
                s = substr($0, RSTART, RLENGTH)
                sub(/^CLAUDE_CONFIG_DIR=/, "", s)
                print s
            }
        }
        exit
    ' "$ALIAS_FILE")"
fi
[[ -n "$CONFIG_DIR" ]] || cma_die "alias '$ALIAS_NAME' not found
    in $ALIAS_FILE"

cma_log "removing alias '$ALIAS_NAME' (config dir: $CONFIG_DIR)"

if (( ! NONINTERACTIVE )); then
    # Destructive op: when no terminal is available to confirm from
    # (CI, SSH

    # without a PTY), refuse rather than block or guess – pass
    # --yes to proceed
    # non-interactively.
    cma_can_prompt || cma_die "no terminal to confirm removal; pass
        --yes to proceed non-interactively"
    read -r -p "Proceed? [y/N] " ans < /dev/tty
    [[ "$ans" =~ ^[Yy]$ ]] || cma_die "aborted"
fi

cma_remove_alias "$ALIAS_NAME"
cma_log "alias removed from $ALIAS_FILE"

if [[ -d "$CONFIG_DIR" ]]; then
    if (( DELETE_DIR )); then
        rm -rf -- "$CONFIG_DIR"
        cma_log "deleted $CONFIG_DIR"
    else
        mv -- "$CONFIG_DIR" "${CONFIG_DIR}.removed.${date +
            %Y%m%d%H%M%S}"
        cma_log "archived $CONFIG_DIR -> ${CONFIG_DIR}.removed.*"
    fi
fi

cat <<EOF

[done] account '$ALIAS_NAME' removed.

```

Notes:

- * Shared store `$SHARED_DIR` is untouched – other accounts still work.
- * Reload your shell so the alias change takes effect.

EOF

8.5 `claude-list-accounts.sh`

```
#!/usr/bin/env bash
# claude-list-accounts.sh – Print a status summary of every
# detected
# Claude Code account: which alias maps to it, whether credentials
# are
# present, and whether its shared-state symlinks are intact.

set -euo pipefail

if (( BASH_VERSINFO[0] < 4 )); then
    for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
        [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
    done
    echo "claude-list-accounts requires bash 4+. Install via: brew
        install bash" >&2
    exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

# Build alias -> dir mapping from the managed alias file.
declare -A ALIAS_OF_DIR
if [[ -f "$ALIAS_FILE" ]]; then
    while IFS= read -r line; do
        [[ "$line" =~ ^alias[[:space:]]+([a-zA-Z0-9_-]
            +)=.*CLAUDE_CONFIG_DIR=([^[:space:]]+) ]] || continue
        ALIAS_OF_DIR["${BASH_REMATCH[2]}"]="${BASH_REMATCH[1]}"
    done < "$ALIAS_FILE"
fi

# Items we expect to be symlinks pointing into $SHARED_DIR. NOTE
# (§11.4
```

```

# own-settings): settings.json is DELIBERATELY excluded – each dir
# has its OWN
# real settings.json (per-alias keys stay local), so it is not a
# shared symlink
# and must not be reported as "not linked".
CHECK_LINKS=(projects history.jsonl plugins todos CLAUDE.md)

printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
    "ALIAS" "CONFIG DIR" "CREDS" "LINKS" "NOTES"

mapfile -t accounts < <(cma_detect_accounts)
for dir in "${accounts[@]"; do
    alias_name="${ALIAS_OF_DIR[$dir]:--}"
    if [[ -f "$dir/.credentials.json" ]]; then creds="yes"; else
        creds="no"; fi
    ok=0 total=0 missing=()
    for item in "${CHECK_LINKS[@]"; do
        total=$((total+1))
        if [[ -L "$dir/$item" ]] \
            && [[ "$(cma_realpath "$dir/$item")" == "$(cma_realpath
                "$SHARED_DIR/$item")" ]]; then
            ok=$((ok+1))
        else
            missing+=("$item")
        fi
    done
    notes=""
    (( ${#missing[@]} )) && notes="not linked: ${missing[*]}"
    printf '%-12s  %-45s  %-6s  %-5s  %s\n' \
        "$alias_name" "$dir" "$creds" "$ok/$total" "$notes"
done

echo
echo "Shared store: $SHARED_DIR"
[[ -d "$SHARED_DIR" ]] || echo " (does not exist – run claude-
unify.sh)"
echo "Alias file: $ALIAS_FILE"
[[ -f "$ALIAS_FILE" ]] || echo " (does not exist – run
install.sh)"

```

8.6 claude-rollback.sh

```

#!/usr/bin/env bash
# claude-rollback.sh – Convenience wrapper that calls claude-
# unify.sh with
# --rollback. Restores every .preunify.<timestamp> backup created
# by the
# unification run and archives the shared store out of the way.
#
# After this you can re-run install.sh to start over from scratch.

set -euo pipefail

```

```

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
exec "$LIB_DIR/claude-unify.sh" --rollback "$@"

```

8.7 claude-export-docs.sh

```

#!/usr/bin/env bash
# claude-export-docs.sh – Convert the multi-account markdown doc
to
# matching .html and .pdf siblings. Re-runnable; overwrites
outputs.
#
# Order of preference for PDF: pandoc + weasyprint > pandoc +
wkhtmltopdf
# > chromium --headless print-to-pdf > weasyprint on the generated
HTML.
# HTML always uses pandoc with a self-contained option so the file
is
# portable without external CSS/JS.

set -euo pipefail

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
    _cma_tgt="$(readlink "$_cma_src")"
    case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
        (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

DOC_DIR="${DOC_DIR:-$HOME/Documents}"
MD_FILE="${MD_FILE:-$DOC_DIR/Claude_Multi_Account_Fine_Tuning.md}"
HTML_FILE="${MD_FILE%.md}.html"
PDF_FILE="${MD_FILE%.md}.pdf"
DOCX_FILE="${MD_FILE%.md}.docx"
# Optional Word styling: first existing reference doc wins.
Generate a starter
# with: pandoc -o reference.docx --print-default-data-file
reference.docx

```

```

DOCX_REFERENCE="${DOCX_REFERENCE:-}"

[[ -f "$MD_FILE" ]] || cma_die "markdown not found: $MD_FILE"
cma_require pandoc

# --- Preprocess: expand `<!-- INCLUDE: relative/path -->` markers
# ---
# These markers, placed inside a fenced code block in the source
# markdown,
# get replaced with the contents of the referenced file (relative
# to the
# directory containing the markdown). That keeps the .md small
# while the
# generated HTML/PDF stays self-contained.
# Portable temp file: BSD/macOS mktemp has no --suffix. pandoc
# reads the
# format from --from, so the missing .md extension is irrelevant.
PROCESSED_MD="$(mktemp "${TMPDIR:-/tmp}/cma-export.XXXXXX")"
trap 'rm -f "$PROCESSED_MD"' EXIT

# Use only POSIX/2-arg awk match() (RSTART/RLENGTH + substr): the
# 3-arg
# match($0,re,arr) capture form is a GNU-awk extension and breaks
# on the BSD
# awk that ships with macOS.
awk -v base="$(dirname "$MD_FILE")" '
  /<!-- INCLUDE: / {
    if (match($0, /INCLUDE: [^ ]+ -->/)) {
      tok = substr($0, RSTART, RLENGTH) # "INCLUDE: path -->"
      sub(/^INCLUDE: /, "", tok)
      sub(/ -->$/, "", tok)           # -> "path"
      path = tok
      if (path !~ /\^\/) path = base "/" path
      while ((getline line < path) > 0) print line
      close(path)
    }
    next
  }
  { print }
' "$MD_FILE" > "$PROCESSED_MD"

# --- HTML ---
# --standalone makes a complete HTML doc (head/body), --embed-
# resources
# inlines CSS/JS/images so the file is self-contained.
cma_log "rendering HTML -> $HTML_FILE"
pandoc \
  --from=gfm+yaml_metadata_block \
  --to=html5 \
  --standalone \
  --embed-resources \
  --toc --toc-depth=3 \
  --highlight-style=tango \

```

```

--metadata title="Claude Multi-Account Fine Tuning" \
-o "$HTML_FILE" \
"$PROCESSED_MD"

# --- PDF ---
cma_log "rendering PDF -> $PDF_FILE"

render_pdf_via_pandoc_engine() {
    local engine="$1"
    command -v "$engine" >/dev/null 2>&1 || return 1
    pandoc \
        --from=gfm+yaml_metadata_block \
        --pdf-engine="$engine" \
        --toc --toc-depth=3 \
        --highlight-style=tango \
        -V geometry:margin=0.9in \
        -V mainfont="DejaVu Serif" \
        -V monofont="DejaVu Sans Mono" \
        --metadata title="Claude Multi-Account Fine Tuning" \
        -o "$PDF_FILE" \
        "$PROCESSED_MD"
}

render_pdf_via_chromium() {
    local engine
    for engine in chromium chromium-browser google-chrome chrome; do
        if command -v "$engine" >/dev/null 2>&1; then
            # Use the freshly generated HTML so styling matches.
            "$engine" --headless --no-sandbox --disable-gpu \
                --print-to-pdf="$PDF_FILE" \
                --print-to-pdf-no-header \
                "file://$HTML_FILE" >/dev/null 2>&1
            return $?
        fi
    done
    return 1
}

render_pdf_via_weasyprint_on_html() {
    command -v weasyprint >/dev/null 2>&1 || return 1
    weasyprint "$HTML_FILE" "$PDF_FILE"
}

# Try engines in order; the first that succeeds wins.
if render_pdf_via_pandoc_engine weasyprint 2>/dev/null; then
    cma_log "pdf engine: weasyprint (via pandoc)"
elif render_pdf_via_pandoc_engine wkhtmltopdf 2>/dev/null; then
    cma_log "pdf engine: wkhtmltopdf (via pandoc)"
elif render_pdf_via_weasyprint_on_html 2>/dev/null; then
    cma_log "pdf engine: weasyprint on html"
elif render_pdf_via_chromium 2>/dev/null; then
    cma_log "pdf engine: chromium headless"

```



```

else
    cma_die "no usable PDF engine. Install one of: weasyprint,
            wkhtmltopdf, chromium."
fi

# --- DOCX ---
# pandoc writes .docx natively (no external engine). A reference
# doc supplies
# styling (fonts, headings, code blocks) when provided.
cma_log "rendering DOCX -> $DOCX_FILE"
docx_args=(
    --from=gfm+yaml_metadata_block
    --to=docx
    --toc --toc-depth=3
    --highlight-style=tango
    --metadata title="Claude Multi-Account Fine Tuning"
    -o "$DOCX_FILE"
)
# Auto-discover a reference doc if none was passed explicitly.
if [[ -z "$DOCX_REFERENCE" ]]; then
    for cand in "$DOC_DIR/reference.docx" "$LIB_DIR/assets/
        reference.docx"; do
        [[ -f "$cand" ]] && { DOCX_REFERENCE="$cand"; break; }
    done
fi
[[ -n "$DOCX_REFERENCE" && -f "$DOCX_REFERENCE" ]] &&
    docx_args+=(--reference-doc "$DOCX_REFERENCE")
if pandoc "${docx_args[@]}" "$PROCESSED_MD"; then
    cma_log "docx ok${DOCX_REFERENCE:+ (styled via
        $DOCX_REFERENCE)}"
else
    cma_warn "DOCX render failed (pandoc); HTML + PDF are still
        produced"
fi

ls -lh "$HTML_FILE" "$PDF_FILE" "$DOCX_FILE" 2>/dev/null || ls
    -lh "$HTML_FILE" "$PDF_FILE"

```

8.8 install.sh

```

#!/usr/bin/env bash
# install.sh – One-shot bootstrap for the Claude multi-account
# toolkit.
# Idempotent; safe to re-run after pulling updates.
#
# What it does, in order:
# 1. Detect tooling (jq, rsync, awk, pandoc) and fail fast if
#    missing.
# 2. Symlink every script into ~/.local/bin (creates the dir if
#    needed).
# 3. Ensure ~/.local/bin is on PATH for future shells.
# 4. Create the managed alias file at $ALIAS_FILE and source it
#    from

```

```

# ~/.bashrc and ~/.zshrc (whichever exist).
# 5. Run claude-unify.sh to merge any existing account dirs.
# 6. Run claude-export-docs.sh to refresh the PDF/HTML alongside
# the
# markdown doc.

set -euo pipefail

# macOS ships bash 3.2 which lacks `mapfile`. If a newer bash is
# available
# (Homebrew installs it at /opt/homebrew/bin/bash on Apple
# Silicon, or
# /usr/local/bin/bash on Intel), re-exec with it. Otherwise tell
# the user
# how to install it.
if (( BASH_VERSION[0] < 4 )); then
  for newer in /opt/homebrew/bin/bash /usr/local/bin/bash; do
    [[ -x "$newer" ]] && exec "$newer" "$0" "$@"
  done
  echo
  "claude-toolkit requires bash 4+. Install via: brew
  install bash" >&2
  exit 1
fi

# Resolve LIB_DIR through any symlinks (install.sh symlinks into
# ~/.local/bin).
_cma_src="${BASH_SOURCE[0]}"
while [ -L "$_cma_src" ]; do
  _cma_tgt="$(readlink "$_cma_src")"
  case "$_cma_tgt" in /*) _cma_src=$_cma_tgt ;; *) _cma_src="$
  (dirname "$_cma_src")/$_cma_tgt" ;; esac
done
LIB_DIR="$(cd "$(dirname "$_cma_src")" && pwd)"
unset _cma_src _cma_tgt
# shellcheck source=lib.sh
source "$LIB_DIR/lib.sh"

cma_log "platform: $(cma_os)"

# 1. Required tooling. pandoc is needed only for doc export; the
# rest are
# load-bearing for unify/add-account.
for t in rsync jq awk; do cma_require "$t"; done
command -v pandoc >/dev/null 2>&1 || cma_warn "pandoc not found –
PDF/HTML export will be skipped"

# 1b. Node deps for the optional TOON utility (scripts/toon.mjs,
# and the
# toon_encode.py wrapper that shells out to it). Soft by design:
# the toolkit's
# core (unify/add-account) needs no Node, so a missing npm is a
# warning, not a

```

```

# hard failure. Idempotent – npm install is a no-op once @toon-
# format/toon is
# already present. (node_modules/ is gitignored; this is how the
# dep arrives.)
_cma_repo_root="$(cd "$LIB_DIR/.." && pwd)"
if [[ -f "$_cma_repo_root/package.json" ]]; then
    if command -v npm >/dev/null 2>&1; then
        cma_log "installing Node deps for the TOON utility (npm
        install) ..."
        ( cd "$_cma_repo_root" && npm install --no-audit --no-fund --
        silent ) \
        || cma_warn "npm install failed – scripts/toon.mjs (TOON
        utility) may be unavailable"
    else
        cma_warn "npm not found – scripts/toon.mjs (TOON utility)
        unavailable until you run 'npm install' in
        $_cma_repo_root"
    fi
fi
unset _cma_repo_root

# 2. Symlink the scripts onto PATH.
BIN_DIR="${BIN_DIR:-$HOME/.local/bin}"
mkdir -p "$BIN_DIR"
for f in "$LIB_DIR"/claude-*.sh; do
    name="$(basename "$f" .sh)"
    link="$BIN_DIR/$name"
    if [[ -L "$link" || -e "$link" ]]; then
        if [[ "$(cma_realpath "$link")" != "$(cma_realpath "$f")" ]];
        then
            mv "$link" "${link}.preunify.${date +%Y%m%d%H%M%S}"
            ln -s "$f" "$link"
            cma_log "linked $link -> $f"
        fi
    else
        ln -s "$f" "$link"
        cma_log "linked $link -> $f"
    fi
done

# 3. Make sure ~/.local/bin is on PATH for new shells. We add
# to .bashrc
# and .zshrc once; existing shells need a manual reload.
# shellcheck disable=SC2016 # $HOME/$PATH are intentionally
# unexpanded literals written into rc files
PATH_LINE='export PATH="$HOME/.local/bin:$PATH"'
for rc in "${CMA_RC_FILES[@]}; do
    [[ -f "$rc" ]] || continue
    if ! grep -F -q "$PATH_LINE" "$rc"; then
        printf '\n# Claude multi-account: ensure ~/.local/bin is on
        PATH\n%s\n' "$PATH_LINE" >> "$rc"
        cma_log "added PATH line to $rc"
    fi
done

```

```

# 4. Alias file + rc sourcing.
cma_ensure_alias_file

# Re-register any pre-existing aliases from $HOME/.bashrc into the
# managed
# alias file, then comment out the originals so they're not
# duplicated.
migrate_inline_aliases() {
    local rc="$1"
    [[ -f "$rc" ]] || return 0
    local tmp; tmp="$(mktemp "${TMPDIR:-/tmp}/cma.XXXXXX")"
    local changed=0
    while IFS= read -r line; do
        if [[ "$line" =~ ^alias[[:space:]]+(claude[0-9a-zA-Z_-]
            +)=.*CLAUDE_CONFIG_DIR=([^\[:space:]]+) ]]; then
            local a="${BASH_REMATCH[1]}" d="${BASH_REMATCH[2]}"
            d="${d//\"/}"
            d="${d/#\$HOME/$HOME}"
            d="${d/#~/$HOME}"
            cma_write_alias "$a" "$d"
            printf '# migrated to %s: %s\n' "$ALIAS_FILE" "$line" >>
                "$tmp"
            changed=1
        else
            printf '%s\n' "$line" >> "$tmp"
        fi
    done < "$rc"
    if (( changed )); then
        cp -p "$rc" "${rc}.preunify.${date +%Y%m%d%H%M%S}"
        mv "$tmp" "$rc"
        cma_log "migrated inline claude* aliases in $rc ->
            $ALIAS_FILE"
    else
        rm -f "$tmp"
    fi
}
for rc in "${CMA_RC_FILES[@]}; do
    migrate_inline_aliases "$rc"
done

# 4b. Copy proxy scripts for provider compatibility (e.g. Poe tool
#      format fix).
PROXY_SRC="$LIB_DIR/proxy"
PROXY_DST="$SHARED_DIR/proxy"
if [[ -d "$PROXY_SRC" ]]; then
    mkdir -p "$PROXY_DST"
    cp "$PROXY_SRC"/*.py "$PROXY_DST/" 2>/dev/null && chmod +x
        "$PROXY_DST"/*.py 2>/dev/null
    cma_log "copied proxy scripts to $PROXY_DST"
fi

```

```

# 4c. Provider session-sync hook + an install-time sync (soft –
# the host may
# lack keys/network at install time, so a failure here is non-
# fatal). The hook
# refreshes provider aliases from cache on every new shell (no
# network) and
# kicks a detached full sync when the cache is stale ($11.4.89).
cma_install_session_hook
if [[ -x "$LIB_DIR/claude-providers.sh" ]]; then
    cma_log "running claude-providers sync (install-time; soft)"
    ( "$LIB_DIR/claude-providers.sh" sync ) || cma_warn "provider
        sync skipped (no keys/network?)"
fi

# 5. Unify whatever accounts exist now.
if (( $(cma_detect_accounts | wc -l) > 0 )); then
    cma_log "running claude-unify.sh"
    "$LIB_DIR/claude-unify.sh"
else
    cma_warn "no ~/${ACCOUNT_PREFIX}* dirs detected – add one with
        claude-add-account.sh"
fi

# 6. Refresh docs if pandoc is available.
if command -v pandoc >/dev/null 2>&1 && [[ -f "$HOME/Documents/
    Claude_Multi_Account_Fine_Tuning.md" ]]; then
    cma_log "running claude-export-docs.sh"
    "$LIB_DIR/claude-export-docs.sh" || cma_warn "doc export failed
        (continuing)"
fi

cat <<EOF

[done] claude-multi-account installed.

    Scripts on PATH:  $BIN_DIR/claude-{unify,add-account,remove-
        account,list-accounts,rollback,export-docs}
    Alias file:       $ALIAS_FILE
    Shared store:     $SHARED_DIR

Open a new shell (or run: source
    $ALIAS_FILE) so the aliases load, then:
    claude-list-accounts      # see what's wired up
    claude-add-account        # add another account
                             interactively
EOF

```

8.9 tests/lib/assert.sh

```

#!/usr/bin/env bash
# assert.sh – small assertion helpers for the test suite. Each
# helper

```

```

# emits `[PASS]` or `[FAIL]` lines and increments counters in the
# caller.
# A test file should source this once and then call the helpers
# inside
# `it` blocks.

TESTS_PASSED="${TESTS_PASSED:-0}"
TESTS_FAILED="${TESTS_FAILED:-0}"
TESTS_FAILURES=()
CURRENT_TEST="${CURRENT_TEST:- (unknown)}"

# Mark the start of an individual test case. Use as:  it "merges
# history.jsonl"
it() {
    CURRENT_TEST="$*"
    printf '\n • %s\n' "$CURRENT_TEST"
}

_pass() {
    TESTS_PASSED=$((TESTS_PASSED + 1))
    printf '    \033[32m[PASS]\033[0m %s\n' "$1"
}

_fail() {
    TESTS_FAILED=$((TESTS_FAILED + 1))
    TESTS_FAILURES+=("$CURRENT_TEST :: $1")
    printf '    \033[31m[FAIL]\033[0m %s\n' "$1"
    [[ -n "${2:-}" ]] && printf '        %s\n' "$2"
}

assert_eq() {
    local expected="$1" actual="$2" msg="${3:-equality}"
    if [[ "$expected" == "$actual" ]]; then _pass "$msg ($expected)"
    else _fail "$msg" "want=$expected got=$actual"; fi
}

assert_file() {
    local path="$1" msg="${2:-file exists}"
    if [[ -f "$path" ]]; then _pass "$msg: $path"
    else _fail "$msg" "missing: $path"; fi
}

assert_dir() {
    local path="$1" msg="${2:-dir exists}"
    if [[ -d "$path" ]]; then _pass "$msg: $path"
    else _fail "$msg" "missing dir: $path"; fi
}

# Portable realpath for the harness. BSD/macOS `readlink` has no
# `-f`, so
# `readlink -f` is a hard error there (empty output) – which would
# make the

```

```

# symlink assertions below pass/fail spuriously on macOS. Self-
# contained
# (mirrors lib.sh's cma_realpath) so assert.sh works even in test
# files that
# don't source lib.sh (e.g. test_add_remove.sh).
_assert_realpath() {
    local p="$1" t dir base
    while [ -L "$p" ]; do
        t="$(readlink "$p")"
        case "$t" in /*) p="$t" ;; *) p="$(dirname "$p")/$t" ;; esac
    done
    dir="$(cd "$(dirname "$p")" 2>/dev/null && pwd -P)" || dir="$(
        (dirname "$p")"
    base="$(basename "$p")"
    printf '%s/%s\n' "$dir" "$base"
}

assert_symlink_to() {
    local link="$1" expected="$2" msg="{3:-symlink target}"
    if [[ ! -L "$link" ]]; then
        _fail "$msg" "not a symlink: $link"
        return
    fi
    local actual; actual="$(_assert_realpath "$link")"
    local want; want="$(_assert_realpath "$expected")"
    if [[ "$actual" == "$want" ]]; then _pass "$msg: $link ->
        $expected"
    else _fail "$msg" "want=$want got=$actual"; fi
}

assert_not_symlink() {
    local p="$1" msg="{2:-not a symlink}"
    if [[ -L "$p" ]]; then _fail "$msg" "is a symlink: $p"
    else _pass "$msg: $p"; fi
}

assert_file_contains() {
    local path="$1" needle="$2" msg="{3:-file contains}"
    if grep -F -q -- "$needle" "$path" 2>/dev/null; then _pass
        "$msg: '$needle' in $(basename "$path")"
    else _fail "$msg" "'$needle' not found in $path"; fi
}

assert_file_not_contains() {
    local path="$1" needle="$2" msg="{3:-file lacks}"
    if grep -F -q -- "$needle" "$path" 2>/dev/null; then _fail
        "$msg" "'$needle' found in $path"
    else _pass "$msg: '$needle' absent from $(basename "$path")"; fi
}

assert_jq() {
    local path="$1" expr="$2" expected="$3" msg="{4:-jq check}"

```

```

local actual; actual="$(jq -r "$expr" "$path" 2>/dev/null ||
echo '<jq-error>')"
if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expr =
$expected)"
else _fail "$msg" "expr=$expr want=$expected got=$actual"; fi
}

assert_lines() {
local path="$1" expected="$2" msg="${3:-line count}"
local actual; actual="$(wc -l < "$path" 2>/dev/null || echo 0)"
actual="${actual// /}"
if [[ "$actual" == "$expected" ]]; then _pass "$msg ($expected
lines)"
else _fail "$msg" "want=$expected got=$actual lines in $path";
fi
}

assert_exit() {
local expected="$1"; shift
local out; out="$("$@" 2>&1)"; local rc=$?
if (( rc == expected )); then _pass "exit $expected for: $"
else _fail "exit code mismatch" "want=$expected got=$rc cmd=$*
out=$out"; fi
}

summary() {
echo
if (( TESTS_FAILED == 0 )); then
printf '\033[32m✓ %d passed, 0 failed\033[0m\n'
"$TESTS_PASSED"
return 0
fi
printf '\033[31m✗ %d failed, %d passed\033[0m\n'
"$TESTS_FAILED" "$TESTS_PASSED"
printf '\nFailures:\n'
for f in "${TESTS_FAILURES[@]}; do printf '  - %s\n' "$f"; done
return 1
}

```

8.10 tests/lib/sandbox.sh

```

#!/usr/bin/env bash
# sandbox.sh – utilities for running scripts against a temp HOME
# so the
# real ~/.claude state is never touched.
#
# Usage from a test file:
#
#   source "$TESTS_DIR/lib/sandbox.sh"
#   make_sandbox          # creates $SANDBOX_HOME, exports HOME,
#                           etc.
#   make_account acct1    # creates a fake ~/.claude-acct1 inside
#                           sandbox

```



```

# make_account acct2 --plugins
# run_unify          # runs claude-unify.sh against the
#                   sandbox
# cleanup_sandbox    # rm -rf the sandbox (registered with
#                   trap)

SCRIPTS_DIR="${SCRIPTS_DIR:-$HOME/Documents/scripts}"

make_sandbox() {
    SANDBOX_HOME="$(mktemp -d "${TMPDIR:-/tmp}/cma-test.XXXXXX")"
    # Override every env var the toolkit reads from. Pointing HOME
    # at the
    # sandbox is the main switch; the other vars only matter if a
    # caller
    # tests defaults explicitly.
    export HOME="$SANDBOX_HOME"
    export SHARED_DIR="$SANDBOX_HOME/.claude-shared"
    export ALIAS_FILE="$SANDBOX_HOME/.local/share/claude-multi-
account/aliases.sh"
    export DEFAULT_DIR="$SANDBOX_HOME/.claude"
    export ACCOUNT_PREFIX=".claude-"
    export CLAUDE_BIN="/usr/bin/true" # dummy; we never actually
    launch claude
    mkdir -p "$DEFAULT_DIR" "$SANDBOX_HOME/.local/bin"
    trap 'cleanup_sandbox' EXIT
}

cleanup_sandbox() {
    [[ -n "${SANDBOX_HOME:-}" ]&& -d "$SANDBOX_HOME" ] || return 0
    # Safety net: only delete if the path matches what mktemp would
    # have
    # produced (cma-test.* somewhere under a tmpdir root).
    # Different
    # systems put mktemp output in different places (/tmp on most
    # Linux,
    # /tmp/.private/<user>/ on this host, /var/folders/... on
    # macOS) so
    # we anchor on the cma-test. prefix rather than the parent.
    case "${(basename "$SANDBOX_HOME")}" in
        cma-test.*) rm -rf -- "$SANDBOX_HOME" ;;
        *) echo "refusing to rm sandbox at unexpected path:
$SANDBOX_HOME" >&2 ;;
    esac
}

# make_account NAME [--plugins] [--settings JSON] [--history
# "line1|line2"] [--memory KEY:VALUE...]
make_account() {
    local name="$1" with_plugins=0 settings_json="" history_lines=()
    shift
    local memory_entries=() todo_entries=()
    while (( $# )); do
        case "$1" in

```

```

--plugins) with_plugins=1; shift ;;
--settings) settings_json="$2"; shift 2 ;;
--history) IFS='|' read -r -a history_lines <<< "$2"; shift
2 ;;
--memory) memory_entries+=("$2"); shift 2 ;;
--todo) todo_entries+=("$2"); shift 2 ;;
*) echo "make_account: unknown arg $1" >&2; return 1 ;;
esac
done

local dir="$HOME/${ACCOUNT_PREFIX}${name}"
mkdir -p "$dir/projects/-home-test/memory" "$dir/todos" "$dir/
tasks" \
    "$dir/plans" "$dir/file-history" "$dir/paste-cache" \
    "$dir/shell-snapshots" "$dir/session-env" "$dir/
telemetry" \
    "$dir/sessions" "$dir/backups" "$dir/cache" "$dir/
plugins"

# Fake credentials so list-accounts reports CREDs:yes.
printf '{"account":"%s"}\n' "$name" > "$dir/.credentials.json"
printf '{"name":"%s"}\n' "$name" > "$dir/.claude.json"

# A unique transcript so the union-merge step has something to
merge.
printf '{"role":"user","msg":"hi from %s"}\n' "$name" \
    > "$dir/projects/-home-test/${uuidgen 2}/dev/null || echo
    "session-${name}).jsonl"

# Memory + todo content if requested.
local m
for m in ${memory_entries[@]+"${memory_entries[@]}"}; do
    local k="${m%*:}" v="${m#*:}"
    printf -- '%s\n' "$v" > "$dir/projects/-home-test/memory/$
{k}.md"
done
local t
for t in ${todo_entries[@]+"${todo_entries[@]}"}; do
    printf '{"subject":"%s"}\n' "$t" > "$dir/todos/${uuidgen 2}/
dev/null || echo "todo-${name}-${t}).json"
done

# History lines (used to test concat+dedup later).
if (( ${#history_lines[@]} )); then
    printf '%s\n' "${history_lines[@]}" > "$dir/history.jsonl"
fi

# Settings JSON if specified, else a minimal default.
if [[ -n "$settings_json" ]]; then
    printf '%s\n' "$settings_json" > "$dir/settings.json"
else
    printf '{"enabledPlugins":{"%s-plugin":true}}\n' "$name" >
"$dir/settings.json"

```

```

fi

if (( with_plugins )); then
    mkdir -p "$dir/plugins/cache/test-marketplace/test-plugin/
        1.0.0"
    mkdir -p "$dir/plugins/marketplaces/test-marketplace"
    cat > "$dir/plugins/installed_plugins.json" <<EOF
{
    "version": 2,
    "plugins": {
        "test-plugin@test-marketplace": [
            {
                "scope": "project",
                "installPath": "$dir/plugins/cache/test-marketplace/test-
                    plugin/1.0.0",
                "version": "1.0.0"
            }
        ]
    }
}
EOF
    cat > "$dir/plugins/known_marketplaces.json" <<EOF
{
    "test-marketplace": {
        "source": {"source": "github", "repo": "example/test"},
        "installLocation": "$dir/plugins/marketplaces/test-
            marketplace"
    }
}
EOF
fi

echo "$dir"
}

run_unify() {
    "$SCRIPTS_DIR/claude-unify.sh" "$@"
}

run_add_account() {
    "$SCRIPTS_DIR/claude-add-account.sh" "$@"
}

run_remove_account() {
    "$SCRIPTS_DIR/claude-remove-account.sh" "$@"
}

run_list_accounts() {
    "$SCRIPTS_DIR/claude-list-accounts.sh" "$@"
}

run_rollback() {

```

```
"$SCRIPTS_DIR/claude-rollback.sh" "$@"
}
```

8.11 tests/run-all.sh

```
#!/usr/bin/env bash
# run-all.sh – discover every test_*.sh sibling, run each in its
#           own
# subshell, and tally pass/fail.
#
# Exit code is 0 only if every test file's `summary` returned 0.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"
export SCRIPTS_DIR

FILES=()
if (( $# )); then
    for arg in "$@"; do FILES+=("$TESTS_DIR/test_${arg}.sh"); done
else
    # Portable discovery: `mapfile`/`readarray` are bash 4+ and
    # absent on the
    # bash 3.2 that ships with macOS, where this test runner must
    # also run.
    while IFS= read -r _f; do FILES+=("${_f}"); done \
        << (find "$TESTS_DIR" -maxdepth 1 -name 'test_*.sh' -type f |
            sort)
fi

PASSED=0 FAILED=0 FAILED_FILES=()

# "${FILES[@]+...}" guards against an empty array under `set -u`
# on bash 3.2,
# which (unlike bash 4.4+) treats "${FILES[@]}" on an empty array
# as unbound.
for f in "${FILES[@]+"${FILES[@]}"; do
    [[ -f "$f" ]] || { echo "missing: $f" >&2; FAILED=$((
        FAILED+1 )); FAILED_FILES+=("$f"); continue; }
    printf '\n\033[1m==> %s\033[0m\n' "${basename "$f"}"
    if bash "$f"; then
        PASSED=$((PASSED + 1))
    else
        FAILED=$((FAILED + 1))
        FAILED_FILES+=("${basename "$f"}")
    fi
done

echo
echo "====="
```

```

echo "Test files: $((PASSED + FAILED))    passed: $PASSED
      failed: $FAILED"
if (( FAILED )); then
    echo "Failed files:"
    for f in "${FAILED_FILES[@]}; do echo "  - $f"; done
    exit 1
fi
echo "ALL GREEN"

```

8.12 tests/test_lib.sh

```

#!/usr/bin/env bash
# test_lib.sh – unit tests for the helper functions in lib.sh.
set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

# shellcheck source=lib/assert.sh
source "$TESTS_DIR/lib/assert.sh"
# shellcheck source=lib/sandbox.sh
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox
# shellcheck source=../lib.sh
source "$SCRIPTS_DIR/lib.sh"
# lib.sh enables `set -e`. The test harness is intentionally
# tolerant of
# non-zero exits (we assert on them), so turn it back off here.
set +e

it "cma_validate_alias accepts well-formed names"
( set -e; cma_validate_alias "claude3" ); assert_eq 0 $? "claude3"
( set -e; cma_validate_alias "work_acct-1" ); assert_eq 0 $?
  "work_acct-1"

it "cma_validate_alias rejects invalid names"
( cma_validate_alias "3claude" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects digit-leading"
( cma_validate_alias "bad name" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects spaces"
( cma_validate_alias "" >/dev/null 2>&1 ); assert_eq 1 $?
  "rejects empty"

it "cma_suggest_alias starts at claude1 when nothing exists"
suggestion="$(cma_suggest_alias)"
assert_eq "claude1" "$suggestion" "first suggestion"

it "cma_suggest_alias increments past existing claudeN aliases"
cma_write_alias claude1 "$HOME/.claude-acct1"
cma_write_alias claude2 "$HOME/.claude-acct2"
cma_write_alias claude5 "$HOME/.claude-acct5"    # gap

```

```

suggestion="$(cma_suggest_alias)"
assert_eq "claude6" "$suggestion" "skips past highest, not count"

it "cma_write_alias is idempotent – rewriting same alias doesn't
    duplicate"
cma_write_alias claude1 "$HOME/.claude-acct1" # second write
count="$(grep -c '^alias claude1=' "$ALIAS_FILE")"
assert_eq "1" "$count" "one alias line for claude1"

it "cma_remove_alias removes the line"
cma_remove_alias claude5
assert_file_not_contains "$ALIAS_FILE" "alias claude5=" "claude5
    removed"

it "cma_ensure_alias_file sources from the shell rc file"
# lib.sh manages .zshrc on macOS and .bashrc + .zshrc on Linux
(CMA_RC_FILES).
# Assert against the platform-appropriate target, selected the
same way lib.sh
# selects it, so the test is correct on both OSes.
if [[ "$(uname -s)" == "Darwin" ]]; then RC_FILE="$HOME/.zshrc";
    else RC_FILE="$HOME/.bashrc"; fi
touch "$RC_FILE"
rm -f "$ALIAS_FILE"
cma_ensure_alias_file
assert_file "$ALIAS_FILE" "alias file created"
assert_file_contains "$RC_FILE" "source \"$ALIAS_FILE\" \"rc file
    gets source line"

it "cma_detect_accounts skips the shared store"
mkdir -p "$HOME/.claude-shared" "$HOME/.claude-acct1"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="${found[*]:-}"
cond=1; [[ "$joined" == *".claude-acct1"* ]] && cond=0; assert_eq
    0 "$cond" "finds .claude-acct1"
cond=1; [[ "$joined" != *".claude-shared"* ]] && cond=0;
    assert_eq 0 "$cond" "excludes .claude-shared"

it "cma_detect_accounts excludes non-Claude .claude-* dirs
    (e.g. .claude-server-commander)"
# Mimic the real-world false positive seen on mistborn.local: an
MCP server
# config dir whose name happens to start with .claude- but has
only its own
# config files, no Claude markers.
mkdir -p "$HOME/.claude-server-commander"
printf '{}\n' > "$HOME/.claude-server-commander/config.json"
printf '{}\n' > "$HOME/.claude-server-commander/feature-
    flags.json"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="${found[*]:-}"

```

```

cond=1; [[ "$joined" != *.claude-server-commander* ]] &&
    cond=0; assert_eq 0 "$cond" "excludes .claude-server-
commander"
cond=1; [[ "$joined" == *.claude-acct1* ]] && cond=0; assert_eq
    0 "$cond" "still finds the legit empty account"

it "cma_detect_accounts includes a populated account dir even if
    it has foreign config too"
# A real account dir with the Claude marker (projects/) shouldn't
    get
# falsely excluded just because some other file happens to be
    there.
mkdir -p "$HOME/.claude-real/projects"
printf '{}\n' > "$HOME/.claude-real/some-other-tool.json"
found=(); while IFS= read -r _l; do found+=("$_l"); done <
    <(cma_detect_accounts)
joined="{${found[*]}:-}"
cond=1; [[ "$joined" == *.claude-real* ]] && cond=0; assert_eq
    0 "$cond" "finds .claude-real"

it "cma_realpath resolves a symlink chain to its canonical target
    (no readlink -f)"
mkdir -p "$HOME/rp/real"
: > "$HOME/rp/real/file"
ln -s "$HOME/rp/real/file" "$HOME/rp/link1"
ln -s "$HOME/rp/link1" "$HOME/rp/link2"
    # chain: link2 -> link1 -> real/file
got="$ (cma_realpath "$HOME/rp/link2")"
want="$ (cd "$HOME/rp/real" && pwd -P)/file"
assert_eq "$want" "$got" "cma_realpath follows the symlink chain"
# A plain (non-symlink) path canonicalizes to itself.
got2="$ (cma_realpath "$HOME/rp/real/file)"
assert_eq "$want" "$got2" "cma_realpath is identity on a real
    path"

it "no runtime script INVOKES 'readlink -f' (absent on BSD/macOS)"
# Strip comments first so explanatory comments mentioning the flag
    don't count;
# we only care about real invocations.
hits="$ (for f in "$SCRIPTS_DIR"/lib.sh "$SCRIPTS_DIR"/install.sh
    "$SCRIPTS_DIR"/claude-*.sh; do sed 's/#.*//' "$f"; done
    2>/dev/null | grep -c 'readlink -f')"
assert_eq 0 "$hits" "zero 'readlink -f' invocations in runtime
    scripts"

it "no committed proof artifact contains a literal secret"
# Regression guard for the H2 incident: live proof files (opcode
    debug config /
# mcp list) once committed a real API key + a DB connection-string
    password.
# We count *suspect* lines rather than print them, so a failure
    never re-echoes a
# secret into the test log. Provider-key prefixes + URL
    user:password@ are the

```

```

# signatures; redacted placeholders carry the word REDACTED and
# are excluded.
proof_dir="$SCRIPTS_DIR/tests/proof"
if [[ -d "$proof_dir" ]]; then
    leaks="$(grep -rIE \
        -e '(sk-ant-|sk-|gsk-|xai-|hf-|AIza|xoxb-|xoxp-|xoxs-|pc-|re-|
            secret_|ghp_|github_pat_|AKIA)[A-Za-z0-9_-]{12,}' \
        -e '://[^\:\/@ "]+:[^\:\/@ "]{4,}@' \
        -e 'eyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}' \
            "$proof_dir" 2>/dev/null | grep -vc 'REDACTED' || true)"
    [[ -z "$leaks" ]] && leaks=0
    assert_eq 0 "$leaks" "proof dir free of literal secrets
        (suspect-line count)"
else
    _pass "no proof dir on this host (nothing to scan)"
fi

# cma_merge_claude_json must UNION the projects subtree across
# accounts while
# keeping each account's private auth keys (userID/
# oauthAccount/...) to ITSELF.
# A regression here would either lose sessions (no union) or leak
# credentials
# between accounts (CRITICAL). Locks the property verified by hand
# this session.
it
    "cma_merge_claude_json: private keys stay per-account;
    projects union (no credential leak)"
if command -v jq >/dev/null 2>&1; then
    mj_a="$SANDBOX_HOME/.mrg-a"; mj_b="$SANDBOX_HOME/.mrg-b"; mkdir
        -p "$mj_a" "$mj_b"
    printf '%s\n' '{"userID":"UID-
        A","oauthAccount":"a@x","projects":{"pa":{"v":1}}}' >
        "$mj_a/.claude.json"
    printf '%s\n' '{"userID":"UID-
        B","oauthAccount":"b@x","projects":{"pb":{"v":2}}}' >
        "$mj_b/.claude.json"
    cma_merge_claude_json "$mj_a" "$mj_b" >/dev/null 2>&1
    assert_eq "UID-A" "$(jq -r .userID "$mj_a/.claude.json")"
        "account A keeps its OWN userID after merge"
    assert_eq "UID-B" "$(jq -r .userID "$mj_b/.claude.json")"
        "account B keeps its OWN userID after merge"
    assert_eq '["pa","pb"]' "$(jq -rc '.projects|keys'
        "$mj_a/.claude.json")" "A sees both projects (union)"
    assert_eq '["pa","pb"]' "$(jq -rc '.projects|keys'
        "$mj_b/.claude.json")" "B sees both projects (union)"
    mj_leak=0
    grep -q 'UID-B\b@x' "$mj_a/.claude.json" && mj_leak=1
    grep -q 'UID-A\b@x' "$mj_b/.claude.json" && mj_leak=1
    assert_eq 0 "$mj_leak" "no cross-account credential leak in
        either direction"
else
    _pass "jq absent – skipping cma_merge_claude_json security test"
fi

```


summary

8.13 tests/test_unify.sh

```
#!/usr/bin/env bash
# test_unify.sh – exercises claude-unify.sh:
#   * basic merge of two accounts: all shared items become
#       symlinks
#   * settings.json enabledPlugins is a strict union
#   * history.jsonl is concatenated and de-duplicated
#   * memory files survive merge (newest account wins conflicts)
#   * plugin manifest installPath is rewritten to the shared store
#   * N>2 accounts also work
#   * a second unify run produces the same state (idempotent)
#   * --rollback restores .preunify.* backups

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"
# shellcheck source=../lib.sh
source "$SCRIPTS_DIR/lib.sh" # defines cma_realpath etc. used
                             directly below
# lib.sh enables `set -e`; this harness asserts on failures, so
                             relax it.

set +e

make_sandbox

# Two fixture accounts with overlapping and unique data.
acct1="$(make_account acct1 --plugins \
  --settings '{"enabledPlugins":
    {"a":true,"b":true},"effortLevel":"high"}' \
  --history 'cmd1|cmd2|shared-line' \
  --memory 'user_role:role from acct1' \
  --todo todo-a-1)"
acct2="$(make_account acct2 --plugins \
  --settings '{"enabledPlugins":
    {"b":true,"c":true},"skipDangerousModePermissionPrompt":true}' \
  --history 'cmd3|shared-line|cmd4' \
  --memory 'user_role:role from acct2 (NEWER)' \
  --memory 'project_notes:notes from acct2 only' \
  --todo todo-b-1)"

# Make sure acct2 is the most recently touched so the unifier's
# "last
# account wins on conflicts" rule applies to it.
```

```

touch "$acct2/projects/-home-test/memory/user_role.md"

# shellcheck disable=SC2119 # test intentionally calls run_unify
# with no args
run_unify >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "claude-unify exits 0"

it "every shared item becomes a symlink into the shared store"
# §11.4 own-settings: settings.json is intentionally NOT in this
# list – it is each
# dir's OWN real file, asserted separately below. Everything else
# (incl. the
# plugin CACHE and history) stays a shared symlink.
for item in projects todos tasks plans file-history paste-cache
  plugins \
    shell-snapshots session-env telemetry sessions backups
  cache \
    stats-cache.json history.jsonl; do
  case "$item" in
    stats-cache.json)
      # Only fixtures with this file get the symlink; skip if
      # absent.
      [[ -e "$SHARED_DIR/$item" ]] || continue ;;
  esac
  assert_symlink_to "$acct1/$item" "$SHARED_DIR/$item" "$acct1
    $item linked"
  assert_symlink_to "$acct2/$item" "$SHARED_DIR/$item" "$acct2
    $item linked"
done

it "settings.json is each dir's OWN real file (§11.4 own-
  settings), not a shared symlink"
# Per-alias permissions/model/hooks must never leak, so unify
# gives every dir its
# OWN settings.json – while plugins (cache) + history stay shared
# symlinks (proven
# in the loop above). The propagated enabledPlugins union lives in
# each own copy.
assert_not_symlink "$acct1/settings.json" "acct1 settings.json is
  OWN"
assert_not_symlink "$acct2/settings.json" "acct2 settings.json is
  OWN"
assert_file "$acct1/settings.json" "acct1 own settings.json
  exists"
assert_file "$acct2/settings.json" "acct2 own settings.json
  exists"
assert_jq "$acct1/settings.json" '.enabledPlugins.a' "true"
  "acct1 own settings carries the enabledPlugins union"
assert_jq "$acct1/settings.json" '.enabledPlugins.c' "true"
  "acct1 own settings gained acct2's plugin via the
  propagated union"

it "shared TEMPLATE settings.json enabledPlugins is the union
  {a,b,c}"

```

```

# Unify unions ONLY the enabledPlugins map into the shared
  template so newly-
# enabled plugins propagate to every dir.
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins | keys |
  sort | join(",")' "a,b,c" "union"

it "$11.4 own-settings: per-alias NON-plugin keys stay local – no
  leak to a sibling dir or to shared"
# effortLevel was set ONLY in acct1;
  skipDangerousModePermissionPrompt ONLY in
# acct2. Under own-settings each keeps its own key, neither leaks
  to the other,
# and NEITHER leaks into the shared template (which carries only
  the plugin union).
assert_jq "$acct1/settings.json" '.effortLevel' "high" "acct1
  keeps its OWN effortLevel"
assert_jq "$acct2/settings.json"
  '.skipDangerousModePermissionPrompt' "true" "acct2 keeps
  its OWN permission flag"
assert_jq "$acct1/settings.json"
  '.skipDangerousModePermissionPrompt // "absent"' "absent"
  "acct2's permission flag did NOT leak into acct1"
assert_jq "$acct2/settings.json" '.effortLevel // "absent"'
  "absent" "acct1's effortLevel did NOT leak into acct2"
assert_jq "$SHARED_DIR/settings.json" '.effortLevel // "absent"'
  "absent" "effortLevel did NOT leak to the shared template"
assert_jq "$SHARED_DIR/settings.json"
  '.skipDangerousModePermissionPrompt // "absent"' "absent"
  "permission flag did NOT leak to the shared template"

it "history.jsonl contains every unique line, no duplicates"
expected_lines=5
actual_lines="$(sort -u "$SHARED_DIR/history.jsonl" | wc -l | tr
  -d ' ')"
assert_eq "$expected_lines" "$actual_lines" "5 unique lines"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd1" "cmd1
  present"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd4" "cmd4
  present"

it "memory: newest account's version wins on filename conflict"
assert_file_contains "$SHARED_DIR/projects/-home-test/memory/
  user_role.md" "NEWER" "acct2 wins"
assert_file "$SHARED_DIR/projects/-home-test/memory/
  project_notes.md" "acct2-only memory survives"

it "plugin manifest paths are rewritten to the shared store"
assert_jq "$SHARED_DIR/plugins/installed_plugins.json" \
  '.plugins["test-plugin@test-marketplace"][0].installPath' \
  "$SHARED_DIR/plugins/cache/test-marketplace/test-plugin/1.0.0" \
  "installPath rewritten"
assert_jq "$SHARED_DIR/plugins/known_marketplaces.json" \
  '["test-marketplace"].installLocation' \
  "$SHARED_DIR/plugins/marketplaces/test-marketplace" \
  "installLocation rewritten"

```

```

it "credentials are NOT shared – each account keeps its own"
assert_not_symlink "$acct1/.credentials.json" "acct1 creds local"
assert_not_symlink "$acct2/.credentials.json" "acct2 creds local"
assert_file_contains "$acct1/.credentials.json" "acct1" "acct1
  creds intact"
assert_file_contains "$acct2/.credentials.json" "acct2" "acct2
  creds intact"

it "unify registers claude<N> aliases for every detected account
  that lacks one"
assert_file_contains "$ALIAS_FILE" "alias
  claude1=\"CLAUDE_CONFIG_DIR=$acct1 cma_run\" \"acct1 gets
  claude1 alias"
assert_file_contains "$ALIAS_FILE" "alias
  claude2=\"CLAUDE_CONFIG_DIR=$acct2 cma_run\" \"acct2 gets
  claude2 alias"
alias_count="$(grep -cE '^alias claude[0-9]+=' "$ALIAS_FILE" ||
  true)"
assert_eq 2 "$alias_count" "exactly two claudeN aliases
  registered"

it "unify preserves claude<N> basenames and fills gaps with next
  free claudeN"
# A dir literally named ~/.claude-claude4 should be aliased as
  claude4, not
# renumbered. Other dirs consume the remaining free claude<N>
  slots.
c4="$(make_account claude4)"
zacct="$(make_account zebra)"
run_unify >/dev/null 2>&1
assert_file_contains "$ALIAS_FILE" "alias
  claude4=\"CLAUDE_CONFIG_DIR=$c4 cma_run\" \"claude4 dir
  keeps claude4 alias"
assert_file_contains "$ALIAS_FILE" "alias
  claude3=\"CLAUDE_CONFIG_DIR=$zacct cma_run\" \"zebra dir
  gets next free claude3 alias"
alias_count="$(grep -cE '^alias claude[0-9]+=' "$ALIAS_FILE" ||
  true)"
assert_eq 4 "$alias_count" "four claudeN aliases total (1,2,3,4)"

it "re-running claude-unify is idempotent (settings.json byte-
  stable)"
checksum_before="$(sha256sum "$SHARED_DIR/settings.json" | cut
  -d' ' -f1)"
target_before="$(cma_realpath "$acct1/projects")" # readlink -f
  is unavailable on BSD/macOS
# shellcheck disable=SC2119 # test intentionally calls run_unify
  with no args
run_unify >/dev/null 2>&1
checksum_after="$(sha256sum "$SHARED_DIR/settings.json" | cut -d'
  ' -f1)"
target_after="$(cma_realpath "$acct1/projects")"
assert_eq "$checksum_before" "$checksum_after" "settings.json
  unchanged"

```

```

assert_eq "$target_before" "$target_after" "symlink target
    unchanged"

it "N>2 accounts: adding a third and re-running unifies it too"
acct3="$(make_account acct3 \
    --settings '{"enabledPlugins":{"d":true}}' \
    --history 'cmd5|cmd1')"
# shellcheck disable=SC2119 # test intentionally calls run_unify
# with no args
run_unify >/dev/null 2>&1
assert_symlink_to "$acct3/projects" "$SHARED_DIR/projects" "acct3
    projects linked"
assert_jq "$SHARED_DIR/settings.json" '.enabledPlugins.d' "true"
    "acct3 plugin merged"
assert_file_contains "$SHARED_DIR/history.jsonl" "cmd5" "acct3
    history merged"

it "$§11.4 own-settings PROOF: a per-alias settings.json edit does
    NOT leak into another dir or shared"
# Add a unique per-alias NON-plugin key to acct1's OWN
# settings.json AFTER unify,
# then re-run unify. own-settings must keep it local: acct1
# retains it, acct2
# never gains it, and the shared template never gains it (it
# carries only the
# enabledPlugins union). This is the load-bearing anti-leak
# guarantee of the
# refactor – a real behavioral proof, not a tautology.
jq '.model = "per-alias-model-acct1-only"' "$acct1/settings.json"
    > "$acct1/settings.json.probe" \
    && mv "$acct1/settings.json.probe" "$acct1/settings.json"
# shellcheck disable=SC2119 # test intentionally calls run_unify
# with no args
run_unify >/dev/null 2>&1
assert_jq "$acct1/settings.json" '.model' "per-alias-model-acct1-
    only" "acct1 keeps its OWN per-alias model after unify"
assert_jq "$acct2/settings.json" '.model // "absent"' "absent"
    "acct1's per-alias model did NOT leak into acct2"
assert_jq "$SHARED_DIR/settings.json" '.model // "absent"'
    "absent" "acct1's per-alias model did NOT leak to the
    shared template"
# The plugin union still flows to every dir (plugins ARE meant to
# be shared).
assert_jq "$acct2/settings.json" '.enabledPlugins.a' "true"
    "acct2 still carries the propagated enabledPlugins union"

# — B1 / B2: absorb_default_plugins + link_default_plugin_subdirs

# Use an isolated SHARED_DIR/DEFAULT_DIR/ACCOUNT_PREFIX triple so
# these tests
# don't interfere with the main sandbox state or the rollback
# assertions below.

b1_sd="$SANDBOX_HOME/.claude-shared-b1"
b1_dd="$SANDBOX_HOME/.claude-b1"

```

```

b1_acct="$SANDBOX_HOME/.claude-b1-acct"
mkdir -p "$b1_dd/plugins/cache/foo/1.0.0"
printf 'hello-b1\n' > "$b1_dd/plugins/cache/foo/1.0.0/file.txt"
mkdir -p "$b1_acct"
printf '{"name":"b1"}\n' > "$b1_acct/.credentials.json"
printf '{"name":"b1"}\n' > "$b1_acct/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$b1_acct/settings.json"
SHARED_DIR="$b1_sd" DEFAULT_DIR="$b1_dd" ACCOUNT_PREFIX=".claude-
b1-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "absorb_default_plugins (B1): files under DEFAULT_DIR/plugins/
    cache are absorbed into shared store"
assert_file "$b1_sd/plugins/cache/foo/1.0.0/file.txt" \
    "absorb_default_plugins: file.txt absorbed from DEFAULT_DIR/
    plugins/cache into SHARED_DIR"

it "link_default_plugin_subdirs (B2): DEFAULT_DIR/plugins/cache
    becomes a symlink into shared store"
b2_is_link=1; [[ -L "$b1_dd/plugins/cache" ]] && b2_is_link=0
assert_eq 0 "$b2_is_link" "DEFAULT_DIR/plugins/cache is a symlink
    after unify"
b2_real="$(cma_realpath "$b1_dd/plugins/cache")"
assert_eq "$(cma_realpath "$b1_sd/plugins/cache")" "$b2_real"
    "symlink resolves to SHARED_DIR/plugins/cache"

it "link_default_plugin_subdirs (B2 idempotent): second unify
    creates no additional .preunify.* backup"
b2_pre="$(find "$b1_dd/plugins" -maxdepth 1 -name
    'cache.preunify.*' 2>/dev/null | wc -l | tr -d ' ')"
SHARED_DIR="$b1_sd" DEFAULT_DIR="$b1_dd" ACCOUNT_PREFIX=".claude-
b1-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
b2_post="$(find "$b1_dd/plugins" -maxdepth 1 -name
    'cache.preunify.*' 2>/dev/null | wc -l | tr -d ' ')"
assert_eq "$b2_pre" "$b2_post" "no new .preunify.* backup created
    on second unify run (idempotent)"

# — B4: sync_claude_md seed branches

# (b) DEFAULT_DIR has a real CLAUDE.md; no account CLAUDE.md.
#     sync_claude_md should seed shared CLAUDE.md from
#     DEFAULT_DIR.

b4b_sd="$SANDBOX_HOME/.claude-shared-b4b"
b4b_dd="$SANDBOX_HOME/.claude-b4b"
b4b_acct="$SANDBOX_HOME/.claude-b4b-acct"
mkdir -p "$b4b_dd" "$b4b_acct"
printf 'default-memory\n' > "$b4b_dd/CLAUDE.md"
printf '{"name":"b4b"}\n' > "$b4b_acct/.credentials.json"
printf '{"name":"b4b"}\n' > "$b4b_acct/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$b4b_acct/settings.json"
SHARED_DIR="$b4b_sd" DEFAULT_DIR="$b4b_dd"
ACCOUNT_PREFIX=".claude-b4b-" \

```

```

"$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "sync_claude_md (B4b): DEFAULT_DIR/CLAUDE.md seeds shared
    CLAUDE.md when no account has one"
assert_file_contains "$b4b_sd/CLAUDE.md" "default-memory" \
    "sync_claude_md: DEFAULT_DIR/CLAUDE.md content seeded into
    shared store"

# (c) No DEFAULT_DIR CLAUDE.md; an account has a real CLAUDE.md.
#     sync_claude_md should fall back and seed from the first
#     account's CLAUDE.md.

b4c_sd="$SANDBOX_HOME/.claude-shared-b4c"
b4c_dd="$SANDBOX_HOME/.claude-b4c"
b4c_acct="$SANDBOX_HOME/.claude-b4c-acct"
mkdir -p "$b4c_dd" "$b4c_acct"
# No CLAUDE.md in b4c_dd (the DEFAULT_DIR).
printf 'acct2-memory\n' > "$b4c_acct/CLAUDE.md"
printf '{"name":"b4c"}\n' > "$b4c_acct/.credentials.json"
printf '{"name":"b4c"}\n' > "$b4c_acct/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$b4c_acct/settings.json"
SHARED_DIR="$b4c_sd" DEFAULT_DIR="$b4c_dd"
ACCOUNT_PREFIX=".claude-b4c-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it
    "sync_claude_md (B4c): account CLAUDE.md seeds shared
    CLAUDE.md when DEFAULT_DIR has none"
assert_file_contains "$b4c_sd/CLAUDE.md" "acct2-memory" \

    "sync_claude_md: account CLAUDE.md seeded into shared
    store when DEFAULT_DIR lacks one"

# — R1: history.jsonl merge must not fuse records across a source
#     that lacks a
#     trailing newline. Regression for cat-based concat gluing one
#     file's last line
#     onto the next file's first line.
r1_sd="$SANDBOX_HOME/.claude-shared-r1"
r1_a="$SANDBOX_HOME/.claude-r1-a"
r1_b="$SANDBOX_HOME/.claude-r1-b"
mkdir -p "$r1_a" "$r1_b"
printf '{"n":"r1a"}\n' > "$r1_a/.credentials.json"; printf
    '{"name":"r1a"}\n' > "$r1_a/.claude.json"
printf '{"n":"r1b"}\n' > "$r1_b/.credentials.json"; printf
    '{"name":"r1b"}\n' > "$r1_b/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$r1_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r1_b/settings.json"
# acct a's history has NO trailing newline (the bug trigger); acct
#     b follows it.
printf '{"e":"one"}\n{"e":"two"}' > "$r1_a/history.jsonl"
printf '{"e":"three"}\n{"e":"four"}\n' > "$r1_b/history.jsonl"
SHARED_DIR="$r1_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r1"
ACCOUNT_PREFIX=".claude-r1-" \

```



```

"$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it
    "history.jsonl (R1): a source missing its trailing newline
    does not fuse records"
r1_two=1; grep -qxF '{"e":"two"}' "$r1_sd/history.jsonl" 2>/
dev/null && r1_two=0
r1_three=1; grep -qxF '{"e":"three"}' "$r1_sd/history.jsonl" 2>/
dev/null && r1_three=0
r1_fused=0; grep -qF '"two"}{"e":"three"}' "$r1_sd/history.jsonl"
2>/dev/null && r1_fused=1
assert_eq 0 "$r1_two" "history line {\\"e\\":\\"two\\"} preserved
intact"
assert_eq 0 "$r1_three" "history line {\\"e\\":\\"three\\"} preserved
intact"
assert_eq 0 "$r1_fused" "no fused two+three record"

# — R2: enabledPlugins union must keep "any true" regardless of
account order.
# Regression: rightmost-wins let the lexically-last account's
false clobber a
# plugin an earlier account had enabled.
r2_sd="$SANDBOX_HOME/.claude-shared-r2"
r2_a="$SANDBOX_HOME/.claude-r2-a"
r2_b="$SANDBOX_HOME/.claude-r2-b"
mkdir -p "$r2_a" "$r2_b"
printf '{"n":"r2a"}\n' > "$r2_a/.credentials.json"; printf
'{"name":"r2a"}\n' > "$r2_a/.claude.json"
printf '{"n":"r2b"}\n' > "$r2_b/.credentials.json"; printf
'{"name":"r2b"}\n' > "$r2_b/.claude.json"
# a (lexically first) enables foo; b (lexically LAST, the overlay
account) disables it.
printf '{"enabledPlugins":{"foo":true}}\n' > "$r2_a/
settings.json"
printf '{"enabledPlugins":{"foo":false,"bar":true}}\n' > "$r2_b/
settings.json"
SHARED_DIR="$r2_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r2"
ACCOUNT_PREFIX=".claude-r2-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

it "enabledPlugins (R2): a plugin enabled in any account stays
enabled (any-true union)"
assert_jq "$r2_sd/settings.json" '.enabledPlugins.foo' "true"
"foo stays true despite last account false"
assert_jq "$r2_sd/settings.json" '.enabledPlugins.bar' "true"
"bar from last account present"

# — R3: a single malformed settings.json must not abort the whole
unify.
# Regression: the multi-file jq -s merge ran unguarded under set
-e.
r3_sd="$SANDBOX_HOME/.claude-shared-r3"
r3_a="$SANDBOX_HOME/.claude-r3-a"
r3_b="$SANDBOX_HOME/.claude-r3-b"
mkdir -p "$r3_a" "$r3_b"

```



```

printf '{"n":"r3a"}\n' > "$r3_a/.credentials.json"; printf
    '{"name":"r3a"}\n' > "$r3_a/.claude.json"
printf '{"n":"r3b"}\n' > "$r3_b/.credentials.json"; printf
    '{"name":"r3b"}\n' > "$r3_b/.claude.json"
printf '{"enabledPlugins":{"ok":true}}\n' > "$r3_a/settings.json"
printf '%s\n' '{ this is not valid json' > "$r3_b/settings.json"
SHARED_DIR="$r3_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r3"
    ACCOUNT_PREFIX=".claude-r3-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
r3_rc=$?

it "settings.json (R3): one malformed account file does not abort
    unify"
assert_eq 0 "$r3_rc" "unify still exits 0 despite a malformed
    settings.json"
r3_tmp=0; [[ -e "$r3_sd/settings.json.tmp" ]] && r3_tmp=1
assert_eq 0 "$r3_tmp" "no orphaned settings.json.tmp left behind"
# The valid account's settings must NOT become a dangling symlink
    (silent loss):
# the merge must still write shared from the valid file(s) and
    link to it.
r3_readable=0; [[ -f "$r3_a/settings.json" ]] && r3_readable=1
assert_eq 1 "$r3_readable" "valid account settings.json still
    readable (not a dangling symlink)"
assert_jq "$r3_a/settings.json" '.enabledPlugins.ok' "true"
    "valid account's settings survive a malformed sibling"

# — R4: directory merge must resolve file conflicts by newest
    mtime, not by the
# lexically-last account name. Regression: pass 2 overlaid
    ACCOUNTS[-1]
# (alphabetically last) unconditionally, so a stale lexically-last
    account
# clobbered fresher content from an earlier account.
r4_sd="$SANDBOX_HOME/.claude-shared-r4"
r4_a="$SANDBOX_HOME/.claude-r4-a"
r4_b="$SANDBOX_HOME/.claude-r4-b"
mkdir -p "$r4_a/projects/p/memory" "$r4_b/projects/p/memory"
printf '{"n":"r4a"}\n' > "$r4_a/.credentials.json"; printf
    '{"name":"r4a"}\n' > "$r4_a/.claude.json"
printf '{"n":"r4b"}\n' > "$r4_b/.credentials.json"; printf
    '{"name":"r4b"}\n' > "$r4_b/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$r4_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r4_b/settings.json"
# Same file in both. b (lexically LAST) is STALE; a (lexically
    first) is FRESH.
# Set explicit mtimes (touch -t, portable) so a is strictly newer
    than b.
printf 'OLD-from-b\n' > "$r4_b/projects/p/memory/note.md"
printf 'NEW-from-a\n' > "$r4_a/projects/p/memory/note.md"
touch -t 202001010000 "$r4_b/projects/p/memory/note.md"
touch -t 202601010000 "$r4_a/projects/p/memory/note.md"
SHARED_DIR="$r4_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r4"
    ACCOUNT_PREFIX=".claude-r4-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1

```

```

it "dir merge (R4): freshest file wins conflicts regardless of
    account name order"
assert_file_contains "$r4_sd/projects/p/memory/note.md" "NEW-from-
    a" \
    "newer file from lexically-earlier account wins (not the stale
    lexically-last one)"

# — R5: rollback must not leave a dangling symlink for an item an
    account never
# had. Such items get a shared-store symlink but NO .preunify
    backup, so the
# restore loop never visits them; once SHARED_DIR is moved aside
    they dangle.
r5_sd="$SANDBOX_HOME/.claude-shared-r5"
r5_a="$SANDBOX_HOME/.claude-r5-a"
r5_b="$SANDBOX_HOME/.claude-r5-b"
mkdir -p "$r5_a/projects" "$r5_b/projects" "$r5_b/todos" # only
    b has todos
printf '{"n":"r5a"}\n' > "$r5_a/.credentials.json"; printf
    '{"name":"r5a"}\n' > "$r5_a/.claude.json"
printf '{"n":"r5b"}\n' > "$r5_b/.credentials.json"; printf
    '{"name":"r5b"}\n' > "$r5_b/.claude.json"
printf '{"enabledPlugins":{}}\n' > "$r5_a/settings.json"
printf '{"enabledPlugins":{}}\n' > "$r5_b/settings.json"
SHARED_DIR="$r5_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r5"
ACCOUNT_PREFIX=".claude-r5-" \
    "$SCRIPTS_DIR/claude-unify.sh" >/dev/null 2>&1
SHARED_DIR="$r5_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r5"
ACCOUNT_PREFIX=".claude-r5-" \
    "$SCRIPTS_DIR/claude-unify.sh" --rollback >/dev/null 2>&1

it "rollback (R5): no dangling shared-store symlink left for a
    never-backed-up item"
r5_dangle=0; [[ -L "$r5_a/todos" ]] && r5_dangle=1
assert_eq 0 "$r5_dangle" "a/todos (symlink, no backup) is removed
    on rollback, not left dangling"

# — R6: rollback restores the OLDEST .preunify backup (the true
    original) when a
# path has more than one, deterministically (find order is
    otherwise arbitrary).
r6_sd="$SANDBOX_HOME/.claude-shared-r6"
r6_a="$SANDBOX_HOME/.claude-r6-a"
mkdir -p "$r6_a"
printf '{"n":"r6a"}\n' > "$r6_a/.credentials.json"; printf
    '{"name":"r6a"}\n' > "$r6_a/.claude.json"
# Two backups for the same path; the oldest timestamp holds the
    true original.
printf 'OLDEST-original\n' > "$r6_a/memo.preunify.20200101000000"
printf 'newer-backup\n' > "$r6_a/memo.preunify.20240101000000"
SHARED_DIR="$r6_sd" DEFAULT_DIR="$SANDBOX_HOME/.claude-r6"
ACCOUNT_PREFIX=".claude-r6-" \
    "$SCRIPTS_DIR/claude-unify.sh" --rollback >/dev/null 2>&1

```

```

it "rollback (R6): oldest .preunify backup wins when a path has
    several"
assert_file_contains "$r6_a/memo" "OLDEST-original" "the true
    (oldest) original is restored deterministically"

it "--rollback restores backups and archives the shared store"
# shellcheck disable=SC2119 # test intentionally calls
#     run_rollback with no args
run_rollback >/dev/null 2>&1
assert_dir "$acct1/projects" "acct1 projects restored as real dir"
assert_not_symlink "$acct1/projects" "no longer a symlink"
cond=1; [[ ! -d "$SHARED_DIR" ]] && cond=0; assert_eq 0 "$cond"
    "shared store moved aside"
shared_archive="$(find "$HOME" -maxdepth 1 -name '.claude-
    shared.removed.*' -type d 2>/dev/null | head -1)"
cond=0; [[ -n "$shared_archive" ]] || cond=1; assert_eq 0 "$cond"
    "archive sibling exists"

# Drift guard: claude-add-account wires CMA_SHARED_ITEMS (lib.sh)
#     while
# claude-unify.sh carries its own SHARED_ITEMS. CLAUDE.md
#     documents "keep the two
# lists in sync". They MUST agree EXCEPT for CLAUDE.md, which
#     unify handles
# specially via sync_claude_md() (promotion from ~/.claude /
#     DEFAULT_DIR – a
# generic rsync/symlink cannot do that). This catches accidental
#     future drift
# (a new shared item added to one list only) without forcing the
#     two paths to merge.
it "shared-item lists stay in sync: unify SHARED_ITEMS ==
    CMA_SHARED_ITEMS minus the CLAUDE.md special-case"
_cma_items="$(awk '/^CMA_SHARED_ITEMS=\\(/{f=1} f{print} f&&/\\)/
    {exit}' "$SCRIPTS_DIR/lib.sh" | tr -d '()' | tr ' ' '\n'
    | grep -vE 'CMA_SHARED_ITEMS=|^$|^CLAUDE\\.md$' | sort |
    tr '\n' ' ')"
_uni_items="$(awk '/^SHARED_ITEMS=\\(/{f=1} f{print} f&&/\\)/
    {exit}' "$SCRIPTS_DIR/claude-unify.sh" | tr -d '()' | tr
    ' ' '\n' | grep -vE 'SHARED_ITEMS=|^$' | sort | tr '\n' '
    ')"
assert_eq "$_uni_items" "$_cma_items" "the two shared-item lists
    agree (modulo the intentional CLAUDE.md special-case)"

summary

```

8.14 tests/test_add_remove.sh

```

#!/usr/bin/env bash
# test_add_remove.sh – exercises claude-add-account.sh and
# claude-remove-account.sh end-to-end:
# * add suggests the next free claudeN
# * add creates the dir with the right symlinks
# * add writes a shell-safe alias line
# * add refuses to overwrite an existing dir

```

```

# * add accepts a custom alias name
# * remove drops the alias line
# * remove archives the dir by default, --delete removes it

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Start with two existing accounts and a unified shared store.
make_account acct1 >/dev/null # side-effect only; returned path
                                not needed
make_account acct2 >/dev/null # side-effect only; returned path
                                not needed
# shellcheck disable=SC2119 # test intentionally calls run_unify
                                with no args
run_unify >/dev/null 2>&1

it "claude-add-account --yes uses sensible defaults"
run_add_account --alias claude3 --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
new_dir="$HOME/.claude-claude3"
assert_dir "$new_dir" "config dir created"
assert_symlink_to "$new_dir/projects" "$SHARED_DIR/projects"
    "projects linked"
# §11.4 own-settings: settings.json is each dir's OWN real file
    (NOT a shared
# symlink) so per-alias permissions/model/hooks never leak. It is
    seeded from
# the shared template, so it carries the enabledPlugins map.
    Meanwhile the plugin
# CACHE (plugins/) and history stay SHARED symlinks.
assert_not_symlink "$new_dir/settings.json" "settings.json is OWN
    (not a shared symlink)"
assert_file "$new_dir/settings.json" "own real settings.json
    exists"
assert_jq "$new_dir/settings.json" 'has("enabledPlugins")' "true"
    "own settings.json carries enabledPlugins (seeded from
    template)"
assert_symlink_to "$new_dir/plugins" "$SHARED_DIR/plugins"
    "plugins (cache) still shared"
assert_symlink_to "$new_dir/history.jsonl" "$SHARED_DIR/
    history.jsonl" "history.jsonl still shared"
assert_symlink_to "$new_dir/CLAUDE.md" "$SHARED_DIR/CLAUDE.md"
    "CLAUDE.md linked"
assert_file_contains "$ALIAS_FILE" "alias claude3=" "alias line
    written"

```

```

assert_file_contains "$ALIAS_FILE" "CLAUDE_CONFIG_DIR=$new_dir"
    "alias points at new dir"

it "claude-add-account refuses to overwrite an existing dir"
( run_add_account --alias claude3 --yes >/dev/null 2>&1 )
rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "exits non-zero
    when dir exists"

it "claude-add-account accepts a custom alias name and custom dir"
custom_dir="$HOME/.claude-mywork"
run_add_account --alias mywork --dir "$custom_dir" --yes >/dev/
    null 2>&1
assert_dir "$custom_dir" "custom dir created"
assert_symlink_to "$custom_dir/projects" "$SHARED_DIR/projects"
    "projects linked"
assert_file_contains "$ALIAS_FILE" "alias mywork=" "custom alias
    written"

it "claude-add-account validates alias names"
( run_add_account --alias "bad name" --yes >/dev/null 2>&1 )
rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "rejects bad
    alias"

it "claude-add-account is non-interactive without --yes
    (CMA_NONINTERACTIVE=1)"
export CMA_NONINTERACTIVE=1
run_add_account --alias claude4 >/dev/null 2>&1
rc=$?
unset CMA_NONINTERACTIVE
assert_eq 0 "$rc" "exit 0 without --yes"
assert_dir "$HOME/.claude-claude4" "config dir created via
    defaults"
assert_file_contains "$ALIAS_FILE" "alias claude4="
    "alias written non-interactively"
# Regression (cma_run_provider migration mis-fire): adding more
accounts must
# not chop previously-written alias lines. All earlier aliases
must survive.
assert_file_contains "$ALIAS_FILE" "alias claude3=" "earlier
    claude3 alias survives"
assert_file_contains "$ALIAS_FILE" "alias mywork="
    "earlier mywork alias survives"

it "claude-remove-account --archive moves the dir aside"
run_remove_account --alias claude3 --archive --yes >/dev/null 2>&1
rc=$?
assert_eq 0 "$rc" "exit 0"
cond=1; [[ ! -d "$HOME/.claude-claude3" ]] && cond=0; assert_eq 0
    "$cond" "original dir gone"
archived="$(find "$HOME" -maxdepth 1 -name '.claude-
    claude3.removed.*' -type d 2>/dev/null | head -1)"

```

```

cond=1; [[ -n "$archived" ]] && cond=0; assert_eq 0 "$cond"
    "archived sibling exists"
assert_file_not_contains "$ALIAS_FILE" "alias claude3=" "alias
    line removed"

it "claude-remove-account --delete actually deletes"
run_remove_account --alias mywork --delete --yes >/dev/null 2>&1
cond=1; [[ ! -d "$HOME/.claude-mywork" ]] && cond=0; assert_eq 0
    "$cond" "dir deleted"
cond=1; [[ -z "$(find "$HOME" -maxdepth 1 -name '.claude-
    mywork.removed.*' 2>/dev/null)" ]] && cond=0; assert_eq 0
    "$cond" "no archive sibling"

it "claude-remove-account rejects unknown alias"
( run_remove_account --alias nonexistent --yes >/dev/null 2>&1 )
rc=$?
[[ $rc -ne 0 ]]; assert_eq 0 $? "unknown alias errors out"

summary

```

8.15 tests/test_list.sh

```

#!/usr/bin/env bash
# test_list.sh - claude-list-accounts.sh reports accurate state.

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

make_sandbox

# Empty-host case first.
it "lists nothing when no accounts exist"
# shellcheck disable=SC2119 # test intentionally calls
    run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
cond=1; [[ "$out" == *"ALIAS"* ]] && cond=0; assert_eq 0 "$cond"
    "header always printed"
echo "$out" | grep -E '^-' >/dev/null; rc=$?
cond=$(( rc != 0 ? 0 : 1 )); assert_eq 0 "$cond" "no account rows"

make_account acct1 >/dev/null # side-effect only; returned path
    not needed
make_account acct2 >/dev/null # side-effect only; returned path
    not needed
# shellcheck disable=SC2119 # test intentionally calls run_unify
    with no args
run_unify >/dev/null 2>&1
run_add_account --alias claude3 --yes >/dev/null 2>&1

```

```

it "shows every account dir with creds + link status"
# shellcheck disable=SC2119 # test intentionally calls
  run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") ".claude-acct1"
  "acct1 listed"
assert_file_contains <(printf '%s\n' "$out") ".claude-acct2"
  "acct2 listed"
assert_file_contains <(printf '%s\n' "$out") ".claude-claude3"
  "claude3 listed"
assert_file_contains <(printf '%s\n' "$out") "claude3" "claude3
  alias resolved"

it "reports the shared store path"
# shellcheck disable=SC2119 # test intentionally calls
  run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
assert_file_contains <(printf '%s\n' "$out") "$SHARED_DIR"
  "shared store path printed"

it "flags accounts with missing credentials"
# claude3 was created by add-account; it has no .credentials.json.
# shellcheck disable=SC2119 # test intentionally calls
  run_list_accounts with no args
out="$(run_list_accounts 2>&1)"
line="$(echo "$out" | grep ".claude-claude3" || true)"
cond=1; [[ "$line" == *" no *" ]] && cond=0; assert_eq 0 "$cond"
  "claude3 creds:no"
line1="$(echo "$out" | grep ".claude-acct1" || true)"
cond=1; [[ "$line1" == *" yes *" ]] && cond=0; assert_eq 0
  "$cond" "acct1 creds:yes"

summary

```

8.16 tests/test_export.sh

```

#!/usr/bin/env bash
# test_export.sh - claude-export-docs.sh produces HTML and PDF,
  expands
# <!-- INCLUDE: path --> markers, and the output PDF starts with
  "%PDF-".

set -uo pipefail

TESTS_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SCRIPTS_DIR="$(cd "$TESTS_DIR/.." && pwd)"

source "$TESTS_DIR/lib/assert.sh"
source "$TESTS_DIR/lib/sandbox.sh"

# The doc-export pipeline hard-requires pandoc (cma_require
  pandoc) plus at

```



```

# least one PDF engine. On hosts without that tooling (minimal
# servers, a
# stock macOS), SKIP rather than fail – the feature is genuinely
# unavailable,
# matching how test_opencode/verify_opencode_live SKIP when
# opencode is absent.
# Engine list mirrors claude-export-docs.sh's own probe order.
_have_pdf_engine() {
    local e
    for e in weasyprint wkhtmltopdf chromium chromium-browser
        google-chrome chrome; do
        command -v "$e" >/dev/null 2>&1 && return 0
    done
    return 1
}
if ! command -v pandoc >/dev/null 2>&1 || ! _have_pdf_engine; then
    echo
        "SKIP: doc-export prerequisites (pandoc + a PDF engine)
        not installed"
    exit 0
fi

make_sandbox
mkdir -p "$HOME/Documents/scripts"

# Tiny fixture markdown + a fixture script to include. We point
# the
# export script at this synthetic doc instead of the real one.
fixture_script="$HOME/Documents/scripts/sample.sh"
cat > "$fixture_script" <<'EOF'
#!/usr/bin/env bash
echo "SAMPLE_SCRIPT_BODY_MARKER"
EOF

fixture_md="$HOME/Documents/fixture.md"
cat > "$fixture_md" <<'EOF'
---
title: Fixture
---
# Fixture

Body paragraph above the include.

```bash
<!-- INCLUDE: scripts/sample.sh -->

```

End of fixture. EOF



```
it "MD_FILE env override is honored and HTML is generated"
MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-docs.sh"
>/dev/null 2>&1 rc=$? assert_eq 0 "$rc" "export exits 0"
assert_file "$HOME/Documents/fixture.html" "html generated"
assert_file "$HOME/Documents/fixture.pdf" "pdf generated"
```

```
it "markers are expanded in output" assert_file_contains
"$HOME/Documents/fixture.html"
"SAMPLE_SCRIPT_BODY_MARKER" "include expanded"
assert_file_not_contains "$HOME/Documents/fixture.html" "<!--
INCLUDE:" "markers removed"
```

```
it "PDF starts with the %PDF- magic bytes" head -c 5 "$HOME/
Documents/fixture.pdf" | grep -q '^%PDF-' assert_eq 0 $? "PDF
magic header present"
```

```
it "regeneration is non-destructive (file timestamps move
forward)" MD_FILE="$fixture_md" "$SCRIPTS_DIR/claude-export-
docs.sh" >/dev/null 2>&1 assert_file "$HOME/Documents/
fixture.html" "html still present after rerun" assert_file "$HOME/
Documents/fixture.pdf" "pdf still present after rerun"
```

summary

---

## ## 9. Troubleshooting

Symptom	Likely cause	Fix
`claude1` works, `claude2` shows empty memory	Symlinks weren't created in `~/.claude-milos85vasic2nd/`.	Re-run `claude-unify.sh` – it's idempotent.
New account can't see history	Forgot to reload shell after `claude-add-account.sh`.	`exec \$SHELL -l` or `source ~/.local/share/claude-multi-account/aliases.sh`.
Plugin loads fail with "plugin not found"	Manifest path points outside the shared store.	Re-run `claude-unify.sh` – the path-rewrite pass will fix it.
Both accounts see *each other's* drafts in the queue	`sessions/` is shared and two processes are running at once.	Don't run two accounts concurrently, or remove `sessions` from

```
`SHARED_ITEMS`. |
| `pandoc` fails to find a PDF engine |
No weasyprint/wkhtmltopdf/chromium installed. |
`brew install weasyprint` or `apt install |
weasyprint`. |
| Want to start completely over |
- |
`claude-rollback` then re-run
`install.sh`. |
```

## ## 10. Sharing the ecosystem with OpenCode

The same plugin ecosystem that the multi-account setup unifies can also be

shared with a host-installed [OpenCode](https://opencode.ai).

Claude Code

plugins are runtime-specific (hooks + JS), but their portable contents –

Anthropic-format **\*\*Skills\*\*** (``SKILL.md``), **\*\*MCP servers\*\*** (``mcp.json``), and

the user **\*\*`CLAUDE.md`\*\*** – map directly onto OpenCode's native ``skills.paths``, ``mcp{}``, and ``instructions[]`` config keys.

``claude-opencode-sync`` performs this translation in one idempotent, additive

pass (it never clobbers OpenCode's own providers or servers):

```
```bash
claude-opencode-sync --dry-run --stats # preview
claude-opencode-sync # apply (prior config is
backed up)
```

It scans the plugin cache, parses both the wrapped (`{ "mcpServers": { ... } }`) and bare (`{ name: { ... } }`) `.mcp.json` shapes, expands `$ {CLAUDE_PLUGIN_ROOT}`, deduplicates servers shipped by multiple plugins, and applies a conservative enable policy — enabling only no-auth/no-secret servers by default so OpenCode startup stays fast, while configuring all the rest `enabled:false`, ready to opencode mcp auth. On the reference host this exposes 1,000+ skills and 110+ MCP servers to OpenCode.

End-to-end proof (sandbox suite + live verification against the real OpenCode binary, writing inspectable evidence to `scripts/tests/proof/`):

```
bash scripts/tests/run-proof.sh
```

Full guide, architecture diagrams, and the enable-policy flowchart live in **OpenCode_Integration.md** and `docs/diagrams/`.

11. Provider aliases — running other LLMs through Claude Code

The same alias machinery that powers `claude1..N` also drives **provider aliases**: a Claude Code alias per LLM provider whose key you keep in your keys file, pointed at that provider's strongest model. The command is `claude-providers` and it is **fully dynamic** — no provider, base URL, or model ID is hardcoded.

Data flow (nothing hardcoded)

1. `~/api_keys.sh` provides the API-key *variable names* (read by `grep`, never executed).
2. `models.dev/api.json` (fetched + cached, with graceful offline degrade) provides each provider's endpoint, transport hint (npm), and full model catalog with cost/context/reasoning metadata.
3. `providers_resolve.py` resolves each key into a record: provider id, alias, base URL, transport (native iff the provider speaks the Anthropic API, else router), and a strong + fast model chosen from the metadata.
4. The bundled **LLMsVerifier** submodule (or a lightweight HTTP probe) optionally verifies the key + model before the alias is activated.
5. For each provider: a non-secret env file, a `cma_run_provider <id> alias`, a `~/ .claude-prov-<id>` config dir linking all shared items (so every plugin is available), and the always-on plugin set are generated.

Two tiny editable inputs cover the only things data can't infer: `providers/key-aliases.json` (key-name → provider normalization) and `providers/overrides.json` (per-provider pins, e.g. a short dseek alias or a native `/anthropic` endpoint).

Transports

- **native** — `claude` runs directly with `ANTHROPIC_BASE_URL`, `ANTHROPIC_AUTH_TOKEN`, `ANTHROPIC_MODEL`, `ANTHROPIC_SMALL_FAST_MODEL`.
- **router** — launches through `claude-code-router` (ccr code), which translates Anthropic ↔ OpenAI/Gemini. The provider entry is written into the ccr config with the live key at launch (`chmod 600`), so the toolkit itself never persists secrets.

Safety

Provider config dirs are **excluded from account auto-detection**, so they never merge into real-account auth and never disturb `claude-unify` / `claude-add-account`. Existing `claudeN` accounts keep working unchanged, and new accounts are still created the same way.

During `claude-providers sync --multi`, the API key is passed to `model_verify.py` via the **CMA_PROBE_KEY environment variable** — never as a command-line argument. This means the key never appears in `ps aux` or `/proc/<pid>/cmdline`, preventing accidental secret exposure to other local users or process-monitoring tools. The key is injected per-invocation and is never written to the alias file, the env file, or any repository path.

Commands

```
claude-providers sync          # discover + create/refresh every
    provider alias
claude-providers list          # alias, provider, transport,
    strong/fast model
claude-providers show <id>     # one provider's resolved env
claude-providers remove <id>   # remove alias + env, back up the
    config dir
claude-providers add --from-key VAR --id PROVIDER # register a
    mapping, then sync
```

Session color — auto-applied (v1.10.0)

Each provider session is now tagged with the alias's deterministic color **automatically**. The toolkit writes an agent-color record (`{"type": "agent-color", "agentColor": "<color>", "sessionId": "<id>"}`) into the session's `.jsonl` — the same record Claude Code's `/color` command writes — via `claude-session apply-color`, called by the `cma_run_provider` wrapper. This is the **only** non-interactive way to set the prompt color on the installed Claude Code (2.1.195): `/color` itself is TUI-only and `claude -p '/color x'` is a no-op, with no CLI flag, settings key, or env var. Verified live that the record is written, is idempotent (re-applied only when the color changes), and persists across `claude --resume`. The toolkit cannot observe the TUI, so confirm the prompt-bar rendering visually on first launch. (Documented in `docs/Provider_Aliases_User_Guide.md` and `docs/SESSION_COLOR.md`.)

Full details, overrides, verification, and troubleshooting live in `docs/Provider_Aliases_User_Guide.md`; diagrams in `docs/diagrams/provider-aliases.md`.

12. Per-project auto-session & per-alias color

A **bare** alias launch — native (claude1, claude2, ...) or provider (deepseek, kimi-for-coding, ...) with no arguments — resumes, or the first time creates, **one long-lived Claude session per project root**. The session id is derived deterministically from the project root path (the git working-tree root if you are inside a repo, otherwise \$PWD), so every alias — and every subdirectory of the repo — maps to the *same* session: switch aliases and you continue the same ongoing work.

The session is named in lowercase kebab-case from the project root directory's basename (e.g. `claude_toolkit` → `claude-toolkit`; `Android 15` → `android-15`). Whitespace and special characters are sanitized to hyphens, and leading/trailing separators are trimmed. The name is applied on **both create and resume**, so a session created *without* a name — by an older wrapper or a plain `claude` invocation — finally gets named the next time you bare-launch an alias into it.

The moment you pass any argument (a prompt, `-p`, `--resume`, `--session-id`, any flag), the wrapper steps aside and your arguments go to `claude` verbatim — no auto-session is injected.

Color is auto-applied (v1.10.0). Each alias maps deterministically to one of Claude Code's 8 prompt colors, and the toolkit now applies it for you: on a bare launch the wrapper writes an agent-color record (`{"type": "agent-color", "agentColor": "<color>", "sessionId": "<id>"}`) into the session's `.jsonl` — the exact record Claude Code's `/color` command writes — via `claude-session apply-color`. It runs **before** launch (to colour a resumed session) and **after** exit (to colour a freshly-created one). It is idempotent: the record is re-written only when the color actually changes, so switching aliases re-colors the *same* session and the file never grows unbounded. This is the **only** non-interactive mechanism: `/color` is TUI-only and `claude -p '/color x'` is a no-op — there is no CLI flag, `settings.json` key, or environment variable for it (verified against `claude 2.1.195`). Verified live that the record is written, is idempotent, and persists across `claude --resume`. The toolkit cannot programmatically observe the TUI, so confirm the prompt-bar rendering visually on first launch.

Implemented by `scripts/claude-session.sh`, driven by the `cma_run / cma_run_provider` alias wrappers in `scripts/lib.sh`. Full details, the naming/color tables, and live verification are in **docs/SESSION_COLOR.md**.

13. TOON utility

scripts/toon.mjs (and its Python wrapper scripts/toon_encode.py) encode JSON into **TOON** (Token-Oriented Object Notation) — a compact encoding that cuts token use for structured data in LLM prompts by declaring array fields once. See **docs/TOON_Integration.md**.

Generated 2026-06-28 (OpenCode integration added 2026-06-06; provider aliases added 2026-06-16; runtime sync via `cma_run/claude-sync-state`, `CMA_PROBE_KEY` secret-hygiene model, and `cma_realpath` BSD portability hardening added 2026-06-28). Maintain by editing this markdown file and re-running `claude-export-docs.sh`.